

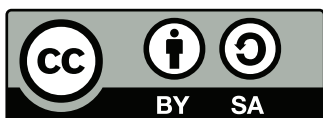


Universidad Complutense de Madrid

Práctica de control de un motor de c. continua

Juan Jiménez

22 de agosto de 2026



El contenido de estos apuntes está bajo licencia Creative Commons Attribution-ShareAlike 4.0
<http://creativecommons.org/licenses/by-sa/4.0/>
©Juan Jiménez

Índice general

1. Descripción del <i>hardware</i> del sistema	2
1.1. Descripción y montaje del sistema experimental	2
1.1.1. El motor EMG30	2
1.1.2. La controladora del motor	6
1.1.3. El microprocesador	9
1.1.4. Conexiones y lectura de las señales de los encoders	11
2. Visual Studio Code: Un IDE para programar la placa	13
2.1. El entorno de desarrollo	13
2.2. Compilación del programa y descarga en la placa del ejecutable	13
3. Descripción del <i>software</i> del sistema ¹	16
3.1. Configuración del microcontrolador (STM32CubeMX)	16
3.2. Programación del Microcontrolador en C	17
3.3. Estructura del Software y Librería CMSIS-DSP	17
3.4. Arquitectura del Bucle de Control	18
3.5. Descripción general de Motor.py	18
4. Motor de continua en lazo abierto. Identificación del motor	22
4.1. Adquisición de datos mediante Motor.py	22
4.2. Modelo e identificación del motor	23
5. Control del motor por realimentación de estados estimados	26
5.1. Un modelo completo del motor	26
5.2. Diseño de un controlador de la posición del motor por realimentación de estados estimados	31
5.2.1. Controlador para el motor ideal	31
5.2.2. Discretización del controlador para control del motor real	35
5.3. Controlador discreto para el modelo completo del motor y el motor real	38

¹Tan solo se describe aquí lo mínimo imprescindible para realizar la práctica. Para una descripción completa consultar el documento `making_of.pdf`

Índice de figuras

1.1. Esquema del motor eléctrico EMG30	3
1.2. Motor eléctrico EMG30	3
1.3. Vistas parciales del mecanismo de la reductora	3
1.4. Vista del encoder situado en la culata del motor EMG30	4
1.5. Señal producida en uno de los encoders (sensor de efecto Hall) al girar el motor	4
1.6. Señales producidas por un encoder en cuadratura al girar el motor	5
1.7. Esquema de la posición relativa de los sensores de efecto Hall (H1 y H2) y los imanes permanentes para el encoder del motor EMG30. En la posición que se muestra, H2 estaría en un flanco e subida o bajada –según la dirección de giro del motor– y H1 Estaría en en centro de un pulso	5
1.8. Esquema del circuito de un puente en H	6
1.9. Señal de PWM	7
1.10. Placa de expansión, X-NUCLEO-IHM04A1	8
1.11. Placa de desarrollo NUCLEO-F411RE	9
1.12. NUCLEO-F411RE. Pin-out compatible con Arduino, la nomenclatura que seguiremos es la de señalada en las etiquetas azules	10
1.13. Esquema de la conexión de una resistencia de <i>pull-up</i> (izquierda) y de la de una resistencia de <i>pull-down</i> (derecha)	11
1.14. vista del montaje completo	12
2.1. Ventana principal de Visual Studio Code	14
2.2. Terminal mostrando el resultado final de la compilación sin errores y menu desplegable Terminal/Run Task	14
2.3. Terminal mostrando en azul el menu desplegable con la tarea flashear STM32	15
3.1. Ventana de visual Studio Code mostrando el árbol de directorios, el archivo motor.py y el botón de ejecución	19
3.2. <i>Dashboard</i> Para el manejo del motor	20
4.1. Circuito equivalente para un motor de continua	23
4.2. Respuesta de un motor de cc a un voltaje de entrada escalón	25
5.1. Esquemas de Simulink para el modelo del motor ideal	26
5.2. Esquema de Simulink del bloque construido para generar una señal de PWM	27
5.3. Bloque para simular una señal de PWM	28
5.4. Modelo del alimentador de tensión al motor incluyendo el cambio de signo de voltaje.	28
5.5. Esquema del modelo incluyendo el motor y su sistema de alimentación	29
5.6. Modelo de Simulink para dos <i>encoders</i> en cuadratura	29
5.7. Modelo completo del motor	30
5.8. Comparación entre las variables de salida posición y velocidad obtenidas por el modelo del motor y el valor de dichas variables obtenidas a partir de la lectura de los <i>encoders</i>	31
5.9. Realimentación de estados para el motor ideal	32
5.10. Realimentación de estados estimados para el motor ideal. La entrada es un escalón de valor 5	33
5.11. Realimentación de estados estimados para el motor ideal	34
5.12. Sistema muestreador/retenedor de orden 0 (ZOH)	35
5.13. Modelo completo del motor con control discretizado por realimentación de estados estimados, acción integral y acción directa	39
5.14. Modelo completo del motor con control discretizado por realimentación de estados estimados, acción integral, acción directa y Anti wind-up.	40

5.15. Sistema completo para alimentar el motor real, empleando el esquema de Simulink (fig. 3.2) para manejar el motor real. Los bloques añadidos antes del motor permiten pasar de tensión en voltios a señal de PWM	41
---	----

Índice de Tablas

1.1. Motor EMG30	2
3.1. Mapa de conexiones nativas entre el sistema electromecánico y la placa STM32 Nucleo.	16
4.1. Estructura del registro de datos del motor (muestra de las primeras filas).	22

Resumen

Estas notas describen el desarrollo completo de un sistema de control para la posición y la velocidad de un motor de continua.

El motor empleado trabaja con un voltaje nominal de 12V. El motor lleva acoplado un encoder en cuadratura, que permite conocer la posición angular relativa de su eje. Además del encoder, el eje del motor lleva también acoplada una reductora que disminuye el número de revoluciones del eje del motor.

La alimentación del motor se regula mediante el uso de transistores de potencia que configuran un puente en H. La tensión suministrada por los transistores se controla mediante una señal de PWM.

Para controlar el motor se emplea una placa de desarrollo programable en C.

Este guión incluye la información necesaria para montar el sistema completo desde cero y poder llevar a cabo una serie de prácticas relacionadas con la identificación, modelado y control del sistema. Cada una de estas dos partes ocupa una sección diferente y ambas pueden leerse de modo independiente.

Capítulo 1

Descripción del *hardware* del sistema

1.1. Descripción y montaje del sistema experimental

1.1.1. El motor EMG30

El EMG30 [1], figura 1.1, es un motor de continua de 12V, equipado con un encoder en cuadratura y una reductora que trasfiere al eje externo una vuelta por cada 30 que da el eje principal. Examinaremos cada uno de estos elementos por separado.

Motor: Se trata de un motor de corriente continua, alimentado mediante escobillas. Las características fundamentales se describen en la tabla 1.1. El motor está pensado para trabajar a un voltaje nominal de 12 V. En la práctica tomaremos dicho voltaje como el valor máximo admisible. Para alimentar el motor emplearemos una controladora que permite variar el voltaje nominal recibido por el motor en el rango 0 – 12V y cuya descripción general se dará más adelante. La figura 1.2 muestra una vista general del motor.

Reductora El eje exterior del motor no coincide con el eje de su rotor. Ambos están conectados entre sí por un juego de ruedas dentadas que permiten desmultiplicar o reducir el número de vueltas. La relación de vueltas entre el eje del rotor y el eje exterior es de 30 : 1, es decir el rotor da 30 vueltas por cada vuelta que observamos en el eje exterior. La reducción en velocidad que supone esta transformación lleva inmediatamente asociado un aumento del par mecánico. Es fácil comprobar este efecto sin más que tratar de girar el eje exterior del motor con la mano y observar la resistencia que ofrece. La figura 1.3 muestra una vista parcial del mecanismo de la reductora.

Encoder Este elemento, situado en la culata del motor, nos permite conocer el ángulo recorrido por el eje del motor. Se trata de un encoder en cuadratura de efecto Hall. En general, un encoder es un dispositivo que permite conocer la posición de un mecanismo, mediante la lectura de pulsos. En nuestro caso, los pulsos los produce un juego de imanes permanentes, situados en el eje del motor, al excitar un par de sensores de efecto Hall. La figura 1.4 Muestra una vista del encoder situado en la culata del EMG30. El disco central unido al eje contiene los imanes y, por tratarse de un encoder en cuadratura, tiene dos sensores de efecto Hall situados a 90° el uno del otro.

Si nos fijamos en el funcionamiento de uno de los sensores de efecto Hall, cada vez que uno de los imanes del disco del eje, pasa por delante del sensor, este se excita produciendo un nivel de tensión alto, cuando el imán deja de excitar al sensor, éste dará un valor de tensión bajo (equivalente a cero). Si el motor esta girando,

Características del motor		Código colores cables	
Voltaje Nominal	12V.	Morado (1)	Encoder B, Vout
Par Nominal	$1,5Kg \cdot m$	Azul (2)	Encoder A, Vout
Velocidad Nominal	$170r.p.m$	verde (3)	Tierra Encoders
Intensidad Nominal	$530mA$	Marrón (4)	Vcc Encoders (3.5v a 20v)
Velocidad sin Carga	$216r.p.m$	Red (5)	V+ Motor
Intensidad sin Carga	$150mA$	Negro (6)	V– Motor
Intensidad bloqueado	2,5A		
Potencia Nominal	4,22W		
Pulsos encoder por vuelta del eje externo	360		

Tabla 1.1: Motor EMG30

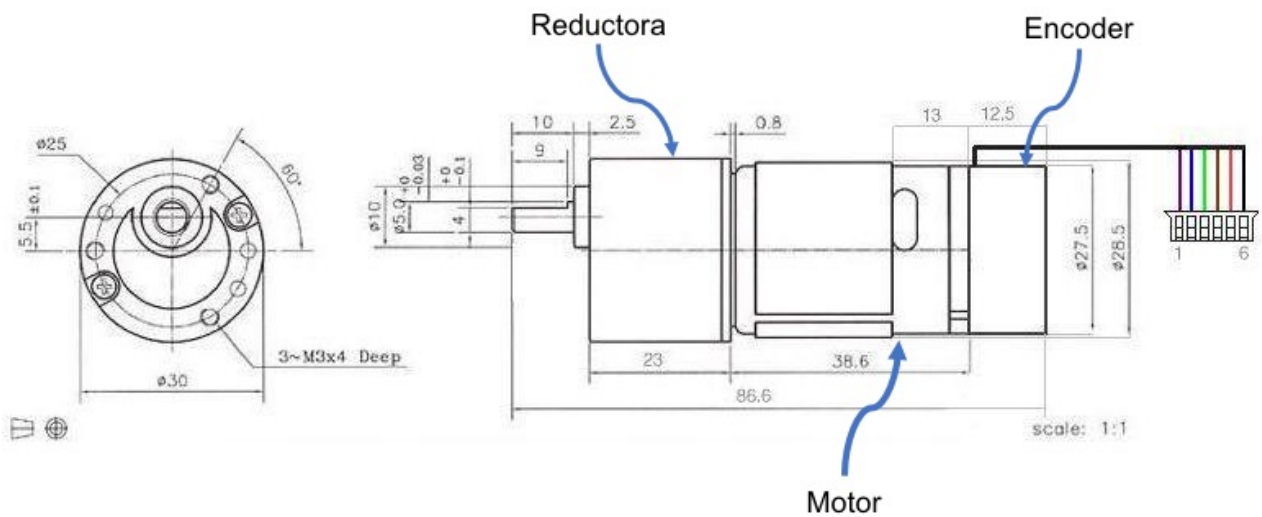


Figura 1.1: Esquema del motor eléctrico EMG30



Figura 1.2: Motor eléctrico EMG30



Figura 1.3: Vistas parciales del mecanismo de la reductora

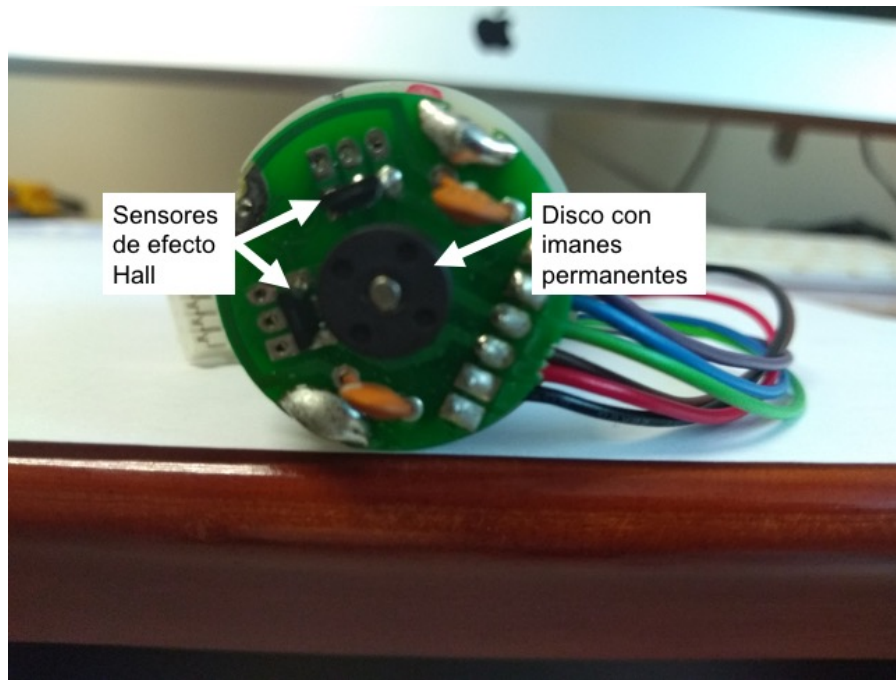


Figura 1.4: Vista del encoder situado en la culata del motor EMG30

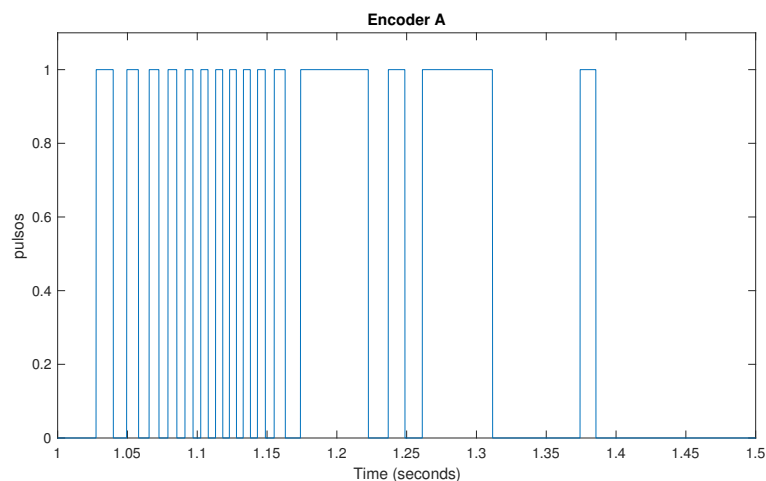


Figura 1.5: Señal producida en uno de los encoders (sensor de efecto Hall) al girar el motor

la señal emitida por el sensor será una onda cuadrada, correspondiente a los niveles altos y bajos de voltaje descritos. La figura 1.5 muestra un ejemplo de la onda generada. Como los imanes permanentes están distribuidos uniformemente entorno al eje del motor, es suficiente con conocer su número para conocer cuánto ha girado el motor. Así, por ejemplo si tenemos tres imanes cada uno cubrirá un rango de 120° por tanto si contamos los flancos de subida de la señal producida y multiplicamos por 120° , podemos calcular los grados que ha avanzado el motor y si dividimos entre tres, podemos calcular el número de vueltas que ha dado. Podemos refinar nuestras medidas un poco más si contamos tantos los flancos de subida como los de bajada. En el ejemplo anterior de los tres imanes, la distancia entre dos flancos consecutivos (subida-bajada), (bajada-subida) será de 60° , con lo que si contamos todos los flancos duplicamos la sensibilidad de nuestro encoder.

Si usamos un solo sensor, como hemos descrito hasta ahora, nos encontramos con el problema de que no sabemos el sentido de giro del motor, ni podemos tampoco detectar los cambios de sentido de giro. La solución a este problema se consigue con el segundo sensor de efecto Hall, que está situado de modo que sus flancos caen justo a mitad de los pulsos del primer encoder. El resultado de las señales de ambos sensores se muestra superpuesto en la figura 1.6.

Si tomamos como referencia el sensor marcado como encoder A (rojo en la figura), si el motor gira en un sentido, sus flancos de subida irán siempre seguidos por flancos de subida del encoder B, pero si gira en sentido contrario entonces un flanco de subida de A irá seguido por un flanco de bajada de B. Además, cuando se de un cambio de sentido de giro observaremos dos flancos seguidos del mismo sensor. Dos sensores así dispuestos

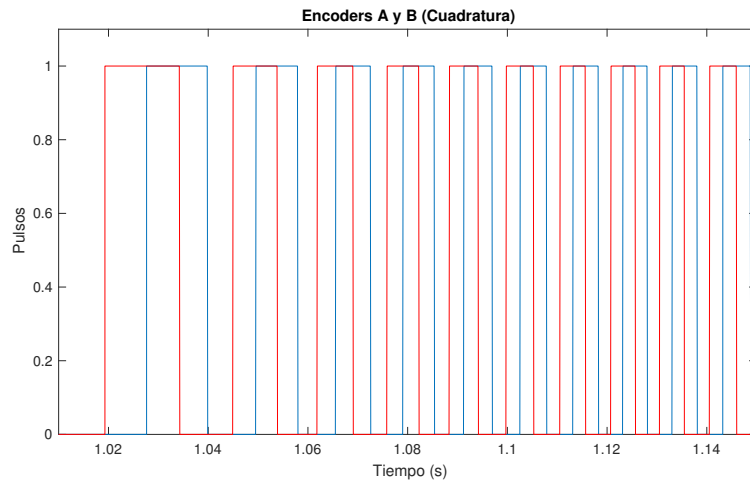


Figura 1.6: Señales producidas por un encoder en cuadratura al girar el motor

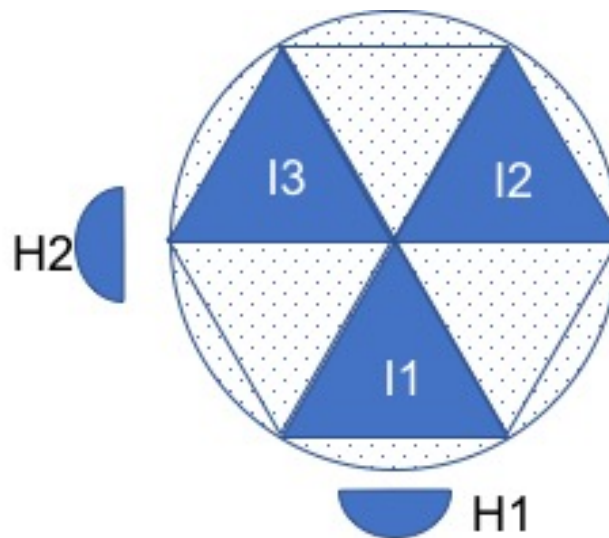


Figura 1.7: Esquema de la posición relativa de los sensores de efecto Hall (H1 y H2) y los imanes permanentes para el encoder del motor EMG30. En la posición que se muestra, H2 estaría en un flanco de subida o bajada –según la dirección de giro del motor– y H1 estaría en el centro de un pulso

constituyen lo que se conoce con el nombre de encoder en cuadratura. Adicionalmente, hemos duplicado el número de flancos, por lo que ahora, la distancia entre flancos es la mitad que para el caso de un solo sensor.

Si nos fijamos en las especificaciones de la tabla 1.1, se indica que el encoder cuenta 360 pulsos por revolución del eje exterior del motor, cada pulso representa un cambio, es decir un flanco de subida o bajada en la señal. La distancia entre dos pulsos sería por tanto de un grado. Esa es la máxima resolución que permite el encoder. Podemos intentar a partir de este dato, deducir la configuración del encoder. Sabemos que la relación de la reductora es 30:1, por tanto el eje del motor da 30 vueltas por cada una de las del eje exterior. Si dividimos el número de pulsos entre el de vueltas del eje principal obtenemos $360/30 = 12$. Es decir el encoder cuenta 12 pulsos por cada vuelta del eje del motor. Como tenemos dos sensores, cada uno de ellos deberá contar seis pulsos, Tres de ellos corresponderán a flancos de subida (s) y tres a flancos de bajada (b) s-b-s-b-s-b. Por tanto podemos deducir que, por cada vuelta del eje, cada sensor se ha excitado tres veces, lo que corresponde a tres imanes equiespaciados en el disco giratorio situado en torno al eje. Cada imán excita al sensor cubriendo un ángulo de 60° , entre los imanes hay una zona *muerta* que cubre también un ángulo de 60° . La disposición de los sensores, situados a 90° , explica su funcionamiento en cuadratura: Cuando un imán está centrado en un sensor, el siguiente imán está empezando a excitar al siguiente sensor, o acaba de excitar al anterior, dependiendo del sentido de giro. La figura 1.7 muestra esquemáticamente este resultado.

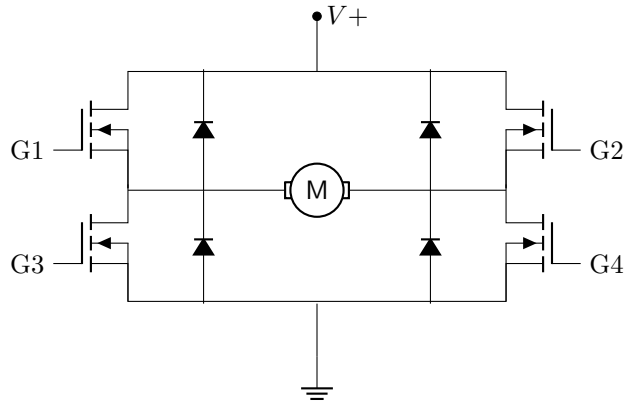


Figura 1.8: Esquema del circuito de un puente en H

1.1.2. La controladora del motor

Para un motor de corriente continua, la velocidad de giro del motor está directamente relacionada con la tensión a la que alimentemos sus terminales eléctricos. Además, el sentido de giro vendrá determinado por la polaridad de la fuente de alimentación. Si invertimos la polaridad de los terminales, el motor invierte su sentido de giro. Podemos, por tanto, variar tanto el sentido de giro como la velocidad. Analicemos por separado ambas posibilidades.

Variación del sentido de giro: Puente en H. Para controlar el sentido de giro del motor es habitual emplear un circuito conocido como puente en H. La figura 1.8 muestra el esquema de un puente en H, construido a partir de transistores. La configuración del circuito, –con el motor situado en el centro de una H formado por los cuatro transistores– es la que da origen a su nombre.

El funcionamiento es sencillo de entender: los transistores actúan como interruptores de modo que si suministramos una tensión de referencia a las puertas G1 y G4 manteniendo a tierra G2 y G3, circulará corriente entre $V+$ y tierra, cruzando el motor desde el terminal izquierdo al derecho, haciendo girar al motor en un sentido. Si, por el contrario, damos tensión a G2 y G3, manteniendo G1 y G4 a tierra, la corriente atravesará el motor desde el terminal derecho al izquierdo, haciendo girar el motor en sentido contrario. Los diodos tienen por objeto proteger el motor durante la conmutación de una situación a otra¹.

Variación de la velocidad: Señal de PWM. Como se ha indicado antes, la velocidad a la que gira un motor de continua varía directamente con el voltaje suministrado. Es más, en un cierto rango de voltaje, y dependiendo del tipo de carga a que esté sometido, se puede considerar que velocidad y voltaje son proporcionales. Podemos por tanto variar la velocidad del motor si somos capaces de variar el voltaje suministrado.

Un método frecuente de hacerlo es emplear una señal de PWM. Las siglas PWM provienen del Inglés *Pulse Width Modulation* (Modulación por Ancho de Pulso). La figura 1.9 Muestra un ejemplo de una señal PWM. La señal tiene tres parámetros característicos importantes. El valor máximo de la señal; en el ejemplo de la figura sería $V_{max} = 10V$. El periodo T , que define el tiempo transcurrido entre dos flancos de subida. Habitualmente, el periodo suele definirse en función del tiempo de respuesta del sistema y se procura que sea significativamente bajo respecto a éste. En el caso del ejemplo mostrado en la figura 1.9 $T = 1ms$. Por último, el Ciclo activo *Duty Cycle* ΔT , que corresponde al periodo durante el cual la señal de PWM está activa y alcanza su valor máximo. En el ejemplo de la figura 1.9 $\Delta T \approx 0,2ms$.

Supongamos que hacemos llegar una señal de PWM a las Puertas G1 y G4 del puente en H de la figura 1.6. Los transistores abrirán el puente durante el *duty cycle* de modo que el motor recibirá corriente solo durante dichos periodos de tiempo. Sin embargo, el tiempo de respuesta de un motor es, por lo general superior al periodo T de la señal, por lo que no podrá arrancar y parar al ritmo de ésta. El resultado es que el motor filtrará la señal quedándose con su valor medio. Si lo calculamos para un ciclo,

$$\frac{1}{T} \int_0^T V_{PWM} dt = \frac{1}{T} \left(\int_0^{\Delta T} V_+ dt + \int_{\Delta T}^T 0 dt \right) = \frac{\Delta T}{T} V_+ \quad (1.1)$$

El motor estaría *viendo* una tensión que equivaldrá al valor de la tensión de la fuente V_+ multiplicada por la relación entre el *duty cycle* y el periodo de la señal de PWM. Si modificamos el *duty cycle* entre 0 y T , El voltaje variará entre 0 y V_+ .

¹El funcionamiento real y el papel de los diodos es un poco más complejo. Aquí solo se han expuesto las ideas más básicas.

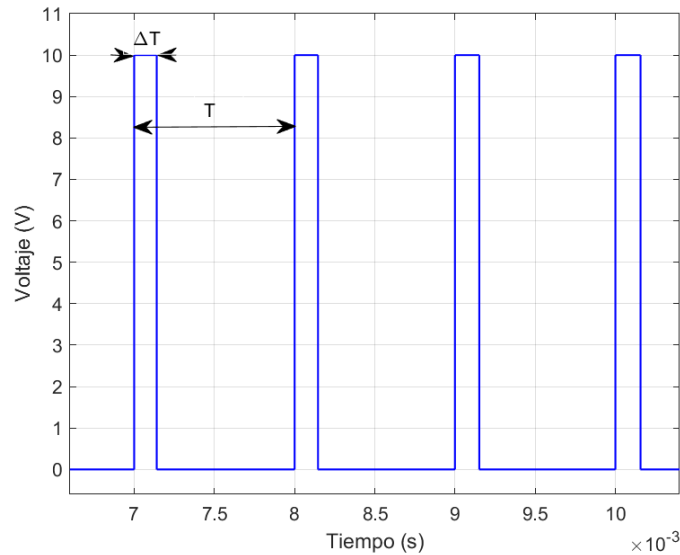


Figura 1.9: Señal de PWM

El controlador X_NUCLEO_IHM15A1 Para controlar el motor emplearemos una placa de desarrollo con un microcontrolador junto con la placa de expansión X_NUCLEO_IHM15A1 fabricadas ambas por la compañía ST [4]. Esta última placa emplea el circuito integrado STSPIN840 [3] que contiene dos puentes en H y todos los dispositivos necesarios para la manipulación y protección de los mismos. A su vez, la placa de expansión viene preparada para su conexión directa con las placas de desarrollo STM32 Nucleo que, como se verá más adelante emplearemos tanto para generar las señales de PWM como para registrar las lecturas de los encoders y controlar el comportamiento del motor. La figura 1.10, muestra una imagen de la placa de expansión.

Entre las características principales del controlador, resalta que admite voltajes de entrada entre 7V y 45V, para corrientes de hasta 1,5A por fase. Como se ha indicado anteriormente, el circuito contiene dos puentes en H, marcados como puente A y puente B. En principio en la placa se puede configurar cada puente por separado, o los dos puentes juntos. De este modo, se podría suministrar corriente a dos motores, con posibilidad de invertir su sentido de giro, o a un solo motor, también con la posibilidad de invertir su sentido de giro. En nuestro caso, emplearemos una configuración en la que alimentaremos un solo motor empleando los dos puentes a la vez, de este modo podemos suministrar una corriente al motor de hasta 3A.

Pero antes de seguir adelante, examinemos la figura 1.10. En la parte superior de la imagen —tal como se ha representado— encontramos una foto de la placa con un conector verde con seis tomas. De abajo arriba, la primera es una entrada, marcada como *GND* que debe conectarse al terminal negativo de la fuente. Corresponde a la *tierra* o referencia de voltaje 0. La segunda toma V_s es la entrada de voltaje, debe conectarse al terminal positivo de la fuente que se empleará para alimentar el motor. Para el caso de nuestro motor debería conectarse al terminal positivo de una fuente de 12V como máximo, puesto que ésta es la tensión nominal del motor (ver sección 1.1.1). Representaría la entrada correspondiente al voltaje $V+$ del puente en H (fig.1.8). Las tomas tercera y cuarta son salidas que suministran voltaje a través de uno de los puentes en H que contiene la placa. En concreto, se trata del puente A. Están marcadas con los símbolos A1 y A2. Por último las tomas quinta y sexta son las salidas de voltaje correspondiente al segundo puente de la placa —el puente B— están marcadas B2, B1.

Como se ha indicado anteriormente, en nuestro caso, vamos a usar ambos puentes a la vez para alimentar al mismo motor. Para ello, (figura 1.10) es preciso colocar el *jumper* de modo paralelo *Parallel mode jumper* en la posición PAR. Esto permite controlar a la vez los dos puentes con una sola señal para el sentido de giro y una sola señal de PWM.

Nuestro motor deberá ir conectado a la placa de modo de las salidas V1 tanto del puente A como del B estén conectadas al cable rojo ($V+$) del motor y las salidas V2 estén conectadas al cable negro ($V-$) del motor.

Por último debemos hablar de los pines de la placa desde los que controlaremos la dirección y la velocidad del motor. Se trata de las entradas lógicas de la tarjeta. En el esquema de la figura 1.10, se han marcado cuatro pines. Los dos en rojo ENA y ENB, Habilitan el puente en H A y el puente en H B, respectivamente. Ambos deberán recibir un 1 lógico desde las correspondientes salidas de la placa del microcontrolador (ver sección 1.1.3). El pin marcado en azul PWMA representa la entrada de la señal de PWM necesaria para controlar el puente A. Dado que hemos colocado el *jumper* en la posición paralelo, esta señal activará a la vez tanto al puente A como al puente B. Esta señal será también enviada desde la placa del microcontrolador. Por último, la entrada marcada en verde como PHA, permite cambiar el par de puertas G1-G4 ó G2-G3 por las que se introduce la

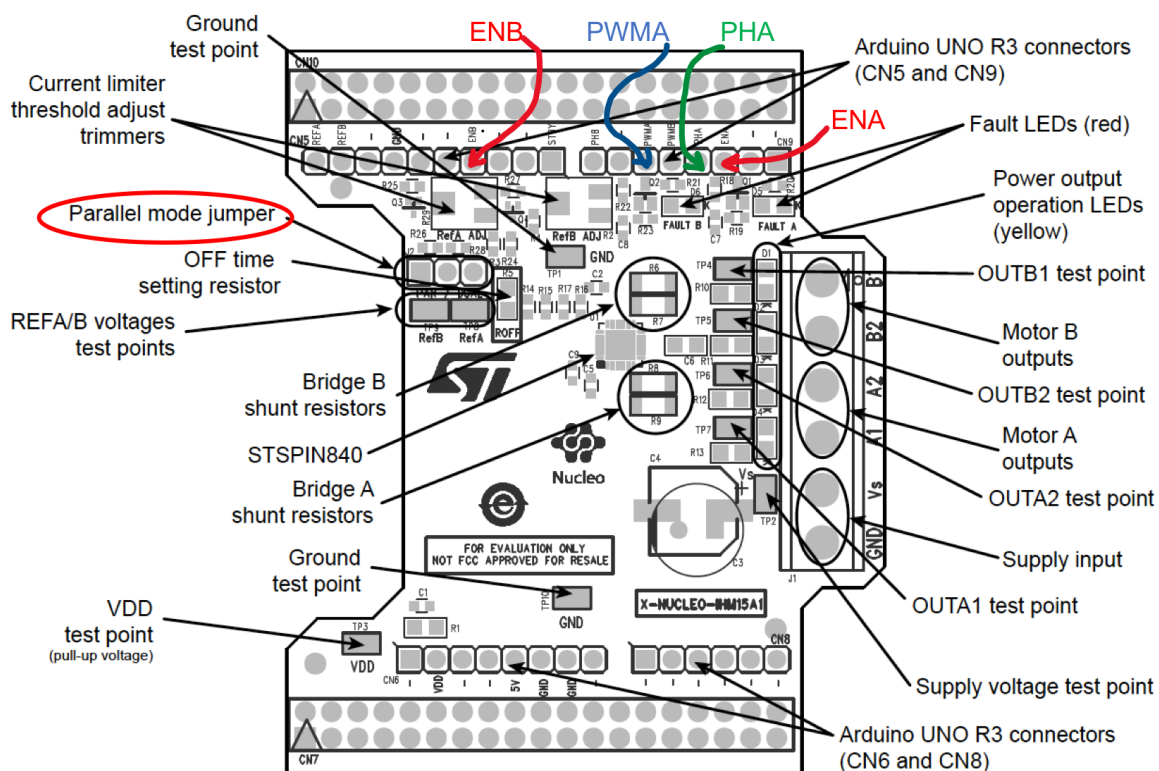


Figura 1.10: Placa de expansión, X-NUCLEO-IHM04A1

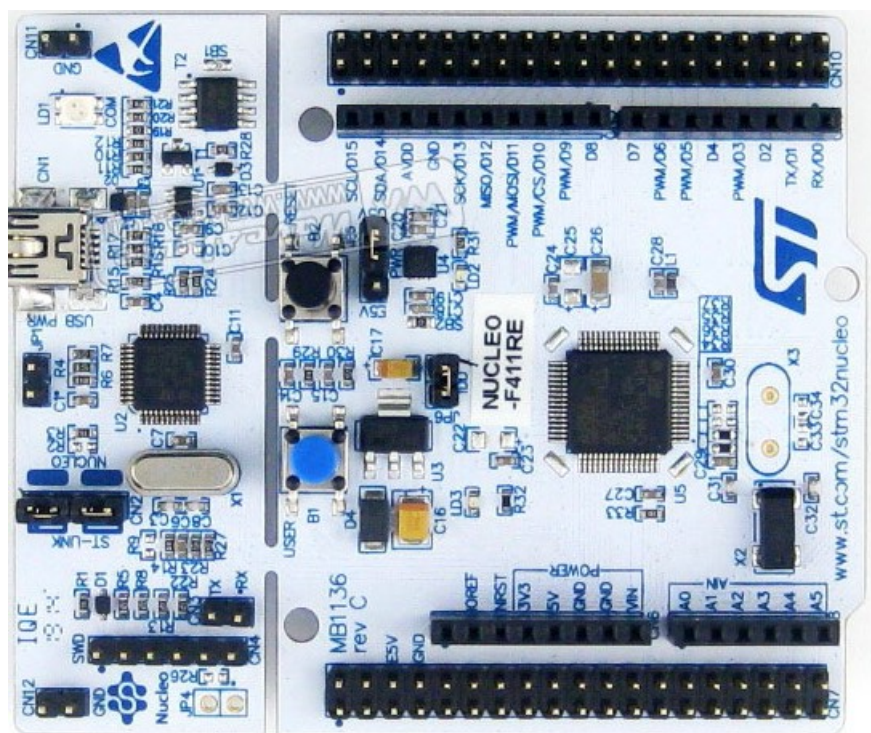


Figura 1.11: Placa de desarrollo NUCLEO-F411RE

señal de PWM, cambiando de este modo el sentido de giro del motor. Para hacerlo girar en un sentido, debemos enviar desde la salida correspondiente de la placa del microcontrolador un 0 lógico, para invertir el sentido de giro, deberemos enviar un 1.

La placa de expansión presenta más opciones de configuración que no emplearemos en estas prácticas. Los interesados en saber más, pueden recurrir a las hojas de características incluidas en la bibliografía

1.1.3. El microprocesador

El siguiente paso es generar las señales de control necesarias para alimentar los pines lógicos de la placa que acabamos de describir. Para ello emplearemos la placa de desarrollo Nucleo-411re STM32 fabricada por la compañía STMicroelectronics. El corazón de la placa es el microcontrolador STM32F411RE, un ARM cortex M-4. La figura 1.11 muestra una imagen de la misma. En dicha imagen, se pueden observar como la placa está dividida en dos partes. La parte de la derecha, antes de llegar a los botones azul y negro, es un programador, conocido como (ST-LINK), que permite introducir programas en la memoria del STM32F411RE, desarrollados en un ordenador normal y compilados mediante el uso de un compilador C/C++ cruzado. El programador se conecta al ordenador mediante el uso de un cable USB estándar. En el extremo de la placa se emplea un conector miniUSB. Además de permitir descargar código en el microcontrolador, el programador permite emular un puerto serie estándar, de modo que se puede emplear para enviar y recibir datos entre la placa y el ordenador en tiempo de ejecución. También permite depurar código, aunque esta opción no la emplearemos en las prácticas.

La parte de la izquierda de la placa, contiene el microcontrolador, un par de botones programables, un par de leds y el resto de la circuitería necesaria para hacer funcionar el microcontrolador. Además presenta dos juegos de pines (*pin-outs*) distintos. El primero de ellos emula el *pin-out* de la famosa placa de desarrollo de Arduino. De este modo, hace posible conectar a la placa del microcontrolador cualquier placa de expansión compatible con la del Arduino. Se trata de los pines hembra que aparecen en la imagen de la figura 1.11. El segundo juego de pines conocido como MORFO ofrece 64 pines macho.

En nuestro caso, emplearemos el *pin-out* compatible con Arduino. Una de las razones para ello es que la placa de expansión que contiene los puentes en H para alimentar el motor, se inserta sobre la placa de desarrollo, empleando dicho *pin-out*. De este modo, podemos identificar rápidamente los pines empleados por la placa de expansión. Por su parte, la placa de expansión vuelve a replicar el *pin-out* de Arduino, con lo que los pines no empleados por la placa de expansión quedan disponibles para otras aplicaciones.

La figura 1.12 muestra el esquema de conexiones del *pin-out* de arduino, así como su nomenclatura. Aunque la específica de Arduino se corresponde con las etiquetas marcadas en verde, nosotros emplearemos habitualmente la correspondiente a MORFO, que se corresponde con las etiquetas marcadas en azul. Si nos fijamos, los pines están agrupados por conectores: CN5 con 10 pines, CN9 con 8 pines, CN6 con 8 pines y CN8 con 6 pines. La

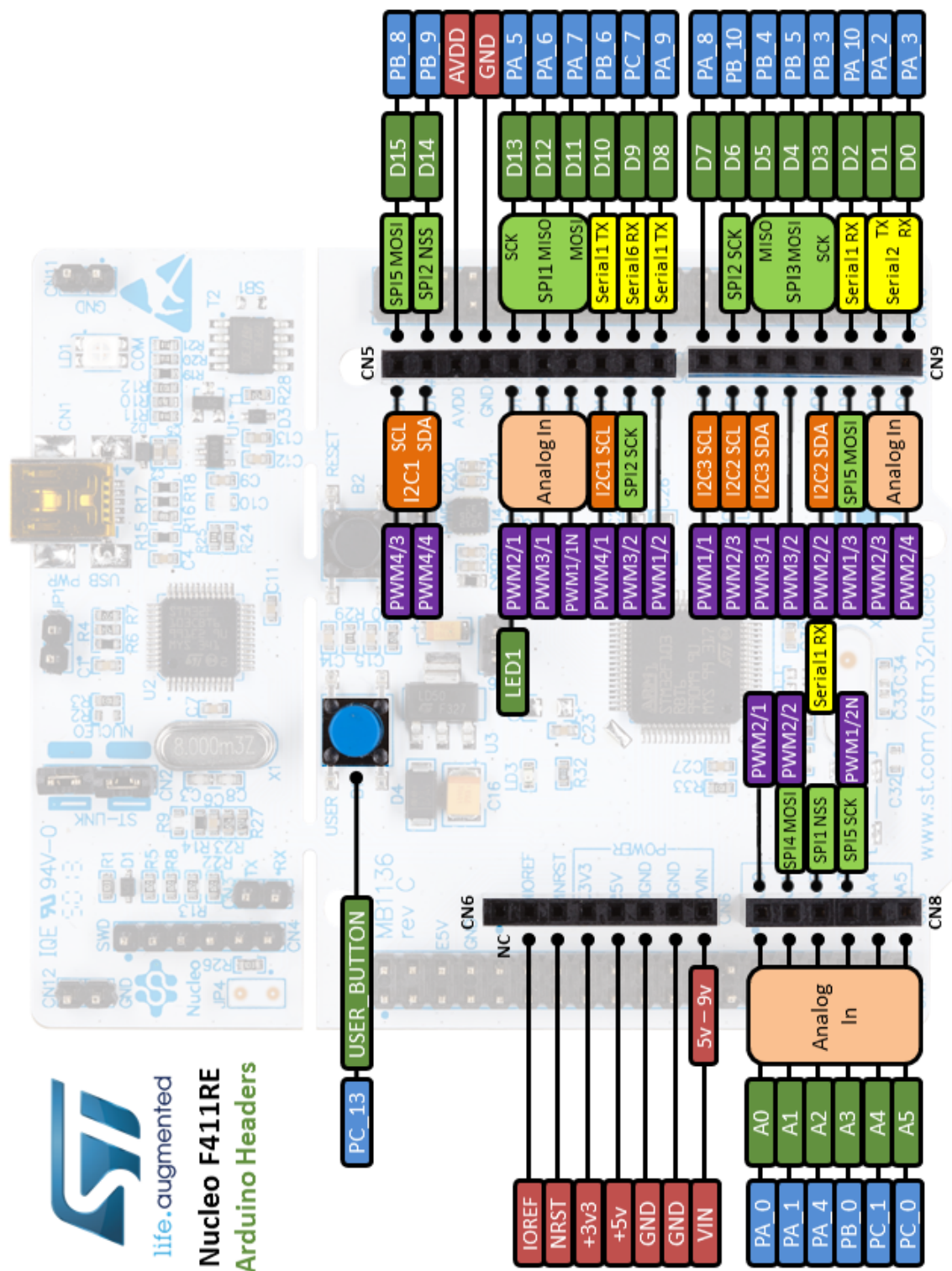


Figura 1.12: NUCLEO-F411RE. Pin-out compatible con Arduino, la nomenclatura que seguiremos es la de señalada en las etiquetas azules

placa de desarrollo empleada es bastante potente y versátil, una descripción completa de la misma excede el propósito de estas páginas. Los interesados en obtener más información pueden consultar el manual de usuario de la placa [2].

1.1.4. Conexiones y lectura de las señales de los encoders

Si comparamos las figuras 1.10 y 1.12 podemos identificar los conectores y los pines empleados por la placa de expansión.

En total, la placa de expansión solo emplea 8 de los 64 pines disponibles. Queda por tanto margen para realizar muchas más lecturas y escrituras de datos desde la placa de desarrollo. La Tabla 3.1 detalla los pines utilizados en esta práctica.

Como se recordará de la sección 1.1.1, el motor EMG30 incorpora un encoder en cuadratura. En primer lugar los encoders deben ser alimentados con un valor de tensión continua en el rango de 3,5 a 20 V. En nuestro caso emplearemos el pin +5V, del conector CN6 para alimentar los encoders. También emplearemos el pin GND, de dicho conector para conectar la tierra de los encoders.

Podemos emplear cualquiera de los pines de entrada/salida disponibles en la placa para leer el valor de dichos encoders. En las especificaciones del motor [1] se indica que las salidas de los encoders son de *colector abierto* y necesitan para trabajar una resistencia de *pull-up*. La figura 1.13 (izquierda) muestra el esquema de la conexión de una resistencia de *pull-up* para una salida de colector abierto. El transistor en la figura actúa como un simple interruptor pasando de corte a saturación al polarizar la base. Si está en corte, el voltaje observado en la salida O valdrá ($V+$). Si está en saturación se leerá un cero en la salida O. Por tanto, es necesario conectar los cables Morado (1) y Azul (2) del motor, descritos en la tabla 1.1, a una entrada de +5V a través de las correspondientes resistencias de *pull-up*.

Por último necesitamos seleccionar unos pines a los que conectar las salidas de los encoders, para monitorizar su valor. En nuestro caso se han seleccionado los pines PA0, PA1. (ver tabla 3.1).

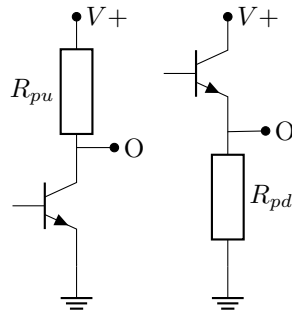


Figura 1.13: Esquema de la conexión de una resistencia de *pull-up* (izquierda) y de la de una resistencia de *pull-down* (derecha)

Un último detalle para terminar con la descripción del *Hardware*. La señal de PWM con la que se activan los puentes en H, desde la placa de desarrollo, debe tener un valor de 0 bien definido. De no ser así, podría suceder que, al parar el motor, éste siguiera moviéndose por encontrar un valor de entrada distinto de 0 e interpretarlo como un 1 lógico. Para asentar bien el valor de cero, se conectará el pin PB4 a una resistencia de *pull-down*. El esquema de dicha conexión se ha representado en la figura 1.13 (derecha). Su funcionamiento es análogo al descrito antes para la resistencia de *pull-up*. Simplemente que ahora el objetivo es asegurar un valor 0 (tierra) cuando el transistor correspondiente esté en corte.

La figura 1.14 muestra una imagen del montaje completo incluyendo el motor, el sistema de control y una placa de inserción. En la imagen puede verse un cable rojo y uno negro que salen de la parte inferior del conector verde de la placa de potencia. Dichos conectores deben conectarse a los terminales + (rojo) y - (negro) de una fuente de alimentación que suministre 12V.

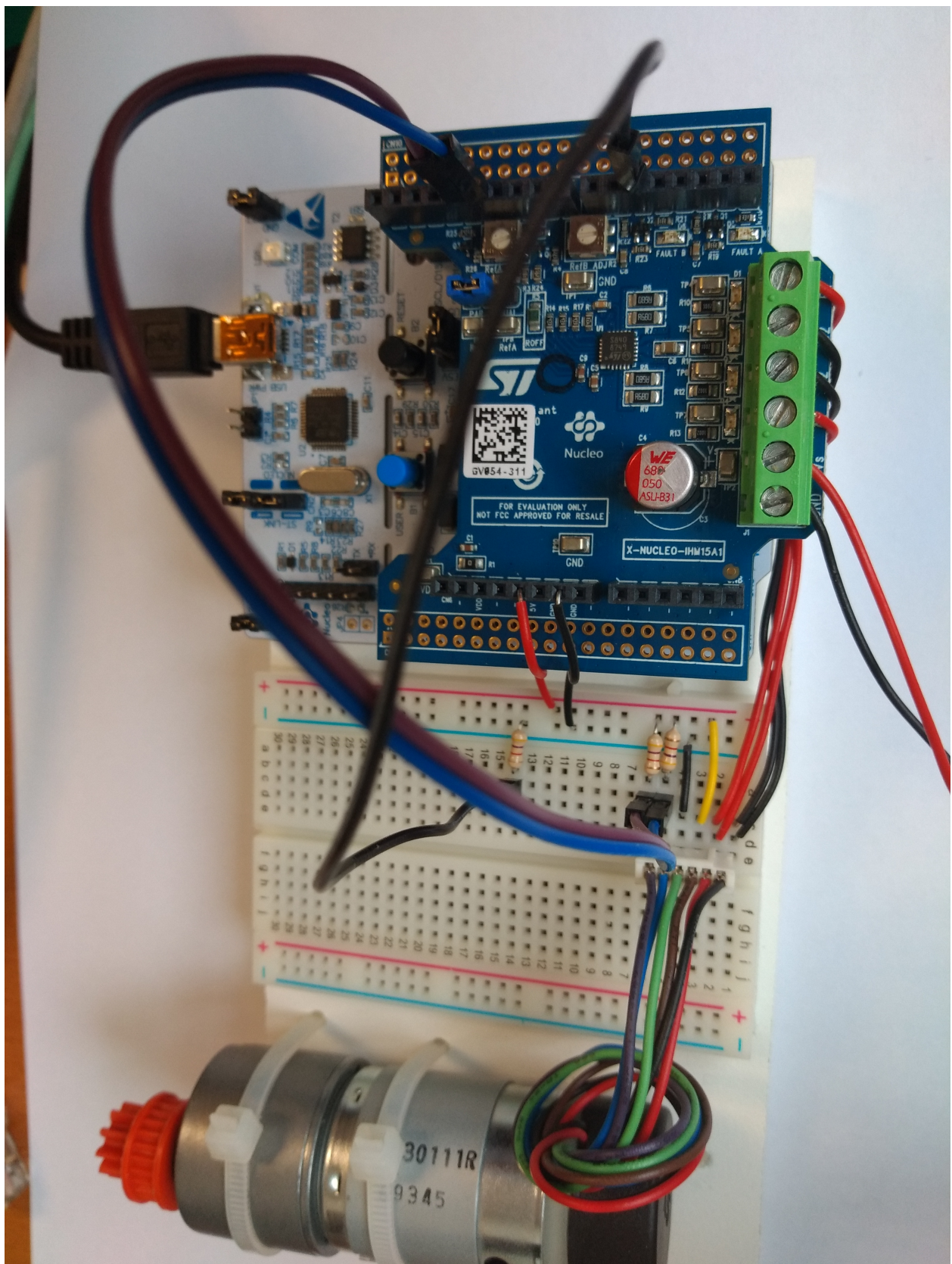


Figura 1.14: vista del montaje completo

Capítulo 2

Visual Studio Code: Un IDE para programar la placa

2.1. El entorno de desarrollo

Antes de hablar del software empleado en la práctica, vamos a describir el entorno de desarrollo (IDE) que emplearemos.

Visual Studio Code es un entorno de desarrollo muy versátil y ampliamente utilizado en programación. Para usarlo es preciso en primer lugar configurarlo adecuadamente de acuerdo a las tareas que se quieran desarrollar. Dar una descripción seria de este entorno está más allá de los objetivos de esta práctica. Diremos simplemente que la versión disponible en los ordenadores del laboratorio ya ha sido configurada adecuadamente para desarrollar y compilar programas en C para la placa de desarrollo, así como para escribir y ejecutar código en Python.

En realidad, el código necesario, tanto el que emplea la placa de desarrollo, escrito en C, como el que usaremos desde el ordenador, escrito en Python. Están preparados de antemano y disponibles en los ordenadores del laboratorio. Para desarrollar el sistema de control, solo serán necesario pequeños cambios en el código suministrado que se describirán en las secciones correspondientes.

Para empezar a trabajar, Abrimos en el ordenador la aplicación Visual Studio Code, el icono es una especie de lazo azul, y debería estar visible en el escritorio de los ordenadores. Haciendo doble clic se nos abrirá una ventana como la de la figura 2.1. La vista no coincidirá exactamente con la que muestra la figura. Para abrir el directorio de trabajo de nuestra práctica, desplegamos el menu **File** arriba a la izquierda de la ventana y buscamos la opción **Open Folder**, buscamos la carpeta `practica_motor_nv` y la abrimos.

Debería aparecer a la izquierda, el mismo árbol de directorios (*Explorer*) que se muestra en la figura 2.1.

De los dos directorios que se muestran bajo `practica_motor_nv`, el primero `guiones` Contiene la documentación de la práctica; como, por ejemplo, este guion. El segundo `motor` es el que contiene el software que vamos a emplear. Solo vamos a hacer referencia a tres ficheros: Dos de ellos están en el directorio `motor/Src` y son los ficheros `main.c` y `motor.c`. El tercero cuelga directamente de `motor` y se llama `motor.py`.

2.2. Compilación del programa y descarga en la placa del ejecutable

Para que el software desarrollado en C, funcione es preciso en primer lugar compilarlo. Para ello, abrimos, por ejemplo el programa `main.c` que es el principal de nuestra aplicación, y buscamos en la barra inferior de la ventana de Visual Studio Code el icono de la rueda dentada y la palabra `build` (ver figura 2.2) lo pulsamos y nos ofrecera las opciones¹ `debug` y `release`. Elegimos `release`. Dejamos que compile el programa y comprobamos que llega al final sin errores (`Build finished with exit code 0`).

Una vez completada la compilación, es preciso descargar el programa ejecutable en la placa de desarrollo. para hacerlo, es preciso tener conectada la placa a uno de los puertos USB del ordenador. En la jerga de los programadores, a la operación de descarga de software se le llama a veces *flashear* la placa. El origen de este término tiene que ver con que el código se almacena en la memoria flash del microcontrolador. Se trata de una memoria no volátil que conserva el programa aun cuando se apague el micro. Cada vez que se vuelve a encender, el micro arranca, busca este código y lo ejecuta.

Para flashear la placa, una vez compilado el programa, abrimos el menú desplegable **Terminal** de la barra superior de la ventana de Visual Studio Code y seleccionamos la opción **Run Task...** (ver figura 2.2). Hacerlo,

¹Solo ofrece estas opciones la primera vez que compilamos, luego lo hace directamente

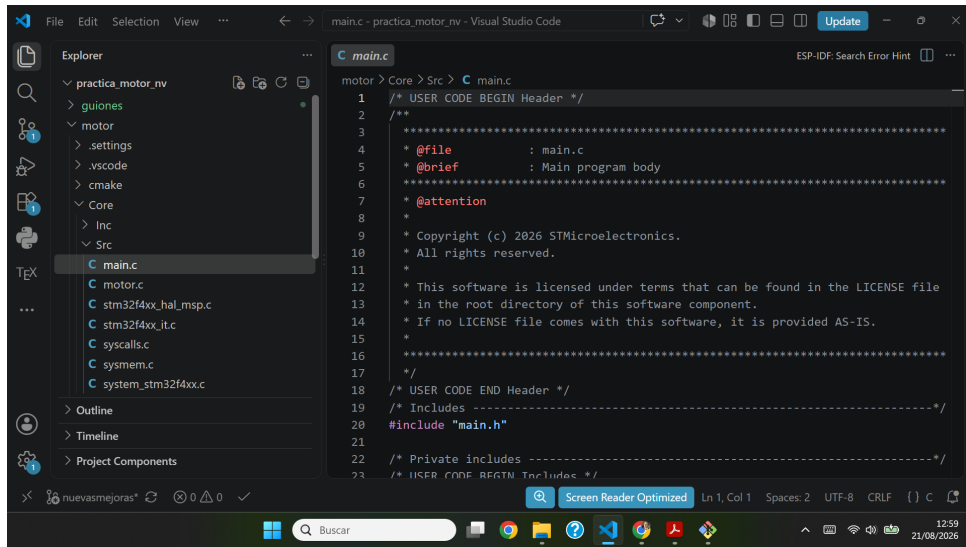


Figura 2.1: Ventana principal de Visual Studio Code

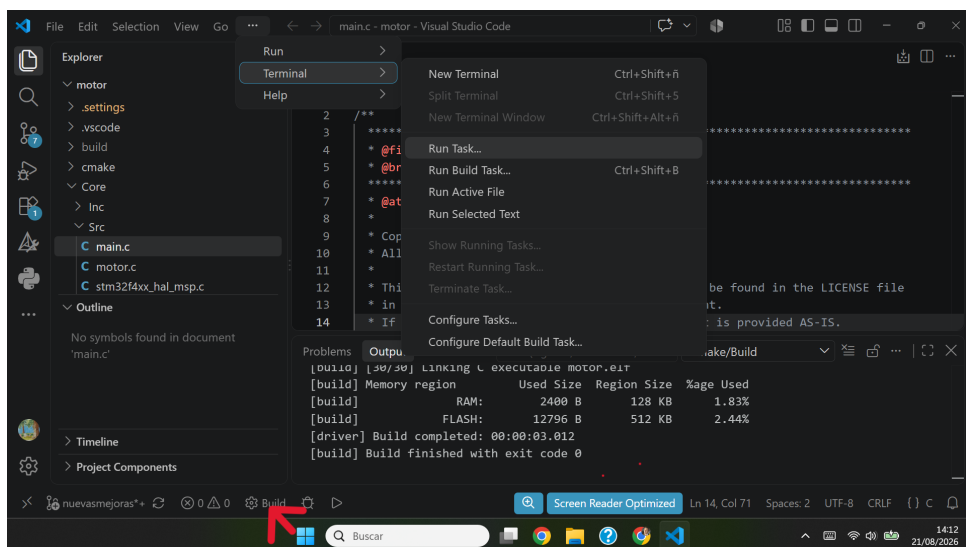


Figura 2.2: Terminal mostrando el resultado final de la compilación sin errores y menu desplegable Terminal/Run Task

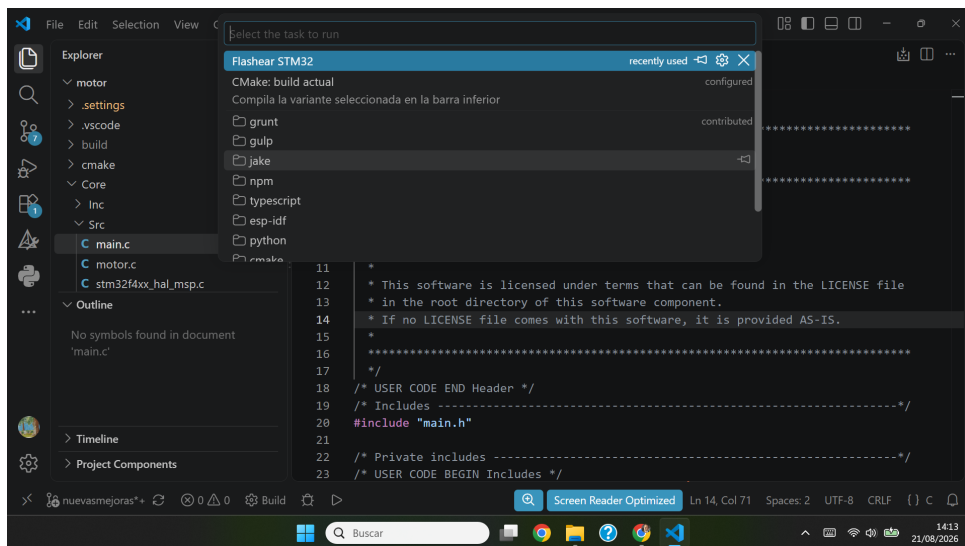


Figura 2.3: Terminal mostrando en azul el menú desplegable con la tarea flashear STM32

se nos abrirá un nuevo menú desplegable en la parte superior de la ventana, con las tareas disponibles. Seleccionamos la opción **flashear STM32** (ver figura 2.3). Si todo va bien, el programa se descargará en la placa. Si se produce un error, lo más probable es que no se haya identificado correctamente el puerto USB. En la mayoría de los casos, este problema se resuelve desenchufando el cable USB y volviendo a enchufarlo.

Una vez completada la descarga, el código comienza a funcionar, y estará esperando instrucciones del ordenador. Ésta se enviarán desde una aplicación desarrollada en Python tal y como se describe en la sección 3.5.

Capítulo 3

Descripción del *software* del sistema ¹

3.1. Configuración del microcontrolador (STM32CubeMX)

El STM32 cuenta con periféricos de hardware dedicados (temporizadores o *Timers*) que pueden encargarse de tareas críticas en tiempo real sin consumir recursos del procesador. Para aprovechar esto, debemos conectar las señales del motor a los pines físicos del microcontrolador que soporten la *Función* requerida. Por ello, la selección de pines realizada para conectar la placa al motor no es casual. La tabla 3.1 resume las conexiones empleadas, y la función específica empleada tanto para la lectura de encoders como para la generación de la señal de PWM

Componente	Señal Motor/Driver	Pin STM32	Función del Periférico (STM32CubeMX)
Encoder	Canal A (Fase A)	PA0	TIM2_CH1 (Modo Encoder)
	Canal B (Fase B)	PA1	TIM2_CH2 (Modo Encoder)
Driver Potencia	PWM (Velocidad)	PA6	TIM3_CH1 (Generación PWM)
	DIR (Sentido)	PB3	GPIO_Output (Salida Digital)
	Habilitar p. H ENA	PA10	GPIO_Output (Salida Digital)
	Habilitar p. H ENB	PA7	GPIO_Output (Salida Digital)

Tabla 3.1: Mapa de conexiones nativas entre el sistema electromecánico y la placa STM32 Nucleo.

Como se deduce de la tabla, la interacción con la planta se delega en gran medida al hardware:

- **Lectura del Encoder (PA0 y PA1):** A diferencia de sistemas más simples donde se programan interrupciones por software para contar los pulsos, aquí conectamos las señales en cuadratura a los canales 1 y 2 del *Timer* 2. Al configurarlo en *Encoder Mode*, el hardware del STM32 detecta automáticamente el sentido de giro y actualiza el registro del contador (CNT). Nuestro software simplemente leerá este registro cuando lo necesite.
- **Generación de PWM (PA6):** Se conecta al canal 1 del *Timer* 3. El microcontrolador generará la señal de control de potencia (*Duty Cycle*) en este pin basándose en el valor que calculemos en nuestro bucle de control y escribamos en su registro de comparación (CCR1).
- **Señal de Dirección (PB3):** A diferencia del PWM, el sentido de giro no requiere una señal de alta frecuencia. Se configura como una salida digital estándar (*GPIO Output*). Escribir un nivel lógico alto (1) hará girar el motor en un sentido, y un nivel bajo (0) en el sentido opuesto, invirtiendo la polaridad en el puente en H.
- **Habilitar ambos puentes en H (PA10, PA7)** Habilitamos los dos puentes en H para alimentar el motor a través de ambos puentes a la vez.

Para interactuar con la planta (el motor de continua), es necesario configurar los periféricos de hardware del microcontrolador que acabamos de describir. Esta tarea se realiza mediante la herramienta gráfica *STM32CubeMX*, la cual genera el código de inicialización (*drivers HAL*). Los elementos fundamentales a configurar son:

- **Lectura del Encoder (TIM2):** Se configura un temporizador en modo *Encoder Mode*. El hardware del STM32 se encargará automáticamente de decodificar las señales en cuadratura (Canal A y Canal B) y actualizar un registro contador (CNT), liberando a la CPU de esta tarea.

¹Tan solo se describe aquí lo mínimo imprescindible para realizar la práctica. Para una descripción completa consultar el documento *making-of.pdf*

- **Generación de PWM (TIM3):** Se configura un temporizador para generar una señal PWM (Modulación por Ancho de Pulso) a una frecuencia superior a 10 kHz para evitar ruido audible y asegurar una dinámica fluida en el puente en H.
- **Bucle de Control (TIM4):** Un temporizador configurado para generar una interrupción periódica exacta cada 1 ms ($T_s = 0,001$ s). Esta interrupción garantiza que nuestro bucle de control se ejecute de forma determinista.
- **Comunicaciones (USART2):** Se habilita el puerto serie asíncrono configurado a 115200 baudios y con interrupciones de recepción (RX) habilitadas, para recibir los comandos desde nuestro ordenador (*Dashboard*) en Python.

3.2. Programación del Microcontrolador en C

No vamos a dar detalles de del manejo de STM32CubeMX. Aquellos que estén interesados en conocer más del proceso de configuración puede encontrar más información en el documento `making_of.pdf`. Simplemente diremos que este programa genera un 'esqueleto del proyecto de software' con el código básico y las librerías necesarias. Para nosotros, el archivo fundamental generado es `main.c`. Este programa contiene la función principal, `main(void)` que se encarga de llamar a todas las funciones de inicialización de los componentes que hemos descrito hasta ahora y ejecuta un bucle infinito `while(1)`

En este bucle `while`, que queda vacío cuando se genera el fichero `main.c` Se ha escrito el código necesario para realizar periódicamente las siguiente tres tareas:

1. Chequear si se ha pulsado el botón azul de la placa, para cambiar de sentido el movimiento del motor.
2. Comprobar, si ha saltado el timer 4, indicando que ha transcurrido 1ms. En caso afirmativo llama a la función `motor_update`, que es la encargada del control del motor.
3. Comprobar si has transcurrido 10ms, en cuyo caso, envía información al ordenador a través del puerto serie.

3.3. Estructura del Software y Librería CMSIS-DSP

Junto al archivo `main.c` Se ha creado de cero un segundo archivo `motor.c` y su correspondiente archivo de cabecera `motor.h`. Este último podemos encontrarlo en el directorio `core/Inc`.

Estos dos archivos son los que contienen todos los elementos necesarios para controlar el motor.

El control moderno, como la realimentación de estados y los observadores de Luenberger, requiere un uso intensivo de álgebra lineal (multiplicación y suma de matrices). Implementar estas operaciones mediante bucles `for` anidados en C es ineficiente y propenso a errores.

Para solucionarlo, emplearemos la librería **CMSIS-DSP** proporcionada por ARM. Esta librería está optimizada en lenguaje ensamblador para aprovechar la Unidad de Coma Flotante (FPU) del procesador Cortex-M4 del STM32, realizando cálculos matriciales en apenas unos pocos ciclos de reloj.

Estas librerías nos van a permitir emplear dos sistemas de control distinto. Uno es un control PID que no emplearemos en estas prácticas el otro es un control por realimentación de estados estimados.

Para mantener el código organizado, encapsularemos todas las variables del motor en una estructura (`struct`) en nuestro archivo de cabecera `motor.h`:

```

1  #include "arm_math.h" // Libreria de procesamiento digital de ARM
2
3  typedef enum {
4      Control_pwm_openloop,
5      Control_position,
6      Control_speed,
7      Control_state_space,
8      Control_speed_state_space
9  } MotorControlMode;
10
11  typedef struct {
12      MotorControlMode mode;
13
14      // Controladores clasicos
15      arm_pid_instance_f32 pid_position;
16      arm_pid_instance_f32 pid_speed;
17
18      // Variables de estado y sensores
19      int32_t posicion_actual;

```

```

20  int32_t posicion_anterior;
21  float32_t velocidad_actual;
22
23  // Control en el Espacio de Estados
24  float32_t x_hat[2];           // Estado estimado [posicion, velocidad]
25  float32_t x_integral_pos;    // Accion integral
26
27  // Referencias
28  int32_t setpoint_position;
29
30  float32_t pwm_output;        // Salida final al hardware
31 } MotorControl;
32
33 extern MotorControl motor;

```

Listing 3.1: Definición de la estructura de control del motor (`motor.h`)

En realidad, para nuestra práctica el único archivo C que debemos modificar es `motor.c`. En concreto, debemos modificar las matrices A , B , C que definen el modelo discreto en variables de estado del motor, así como las ganancias del controlador, del observador y del control integral. El siguiente listado muestra las líneas de código que contienen estas variables. Por defecto, todos los valores están puestos a cero y deben ser sustituidos por los valores calculados para el controlador.

```

16 // Matrices del modelo discretizado (A, B, C)
17 float32_t Ad_f32[4] = {0.0000f, 0.0000f, 0.0000f, 0.0000f}; // Matriz A (2x2)
18 float32_t Bd_f32[2] = {0.0000f, 0.0000f}; // Matriz B (2x1)
19 float32_t Cd_f32[2] = {0.0000f, 0.0000f}; // Matriz C (1x2)
20
21 // Ganancias del controlador y observador de espacio de estados
22 float32_t Lp_f32[2] = {0.0000f, 0.0000f}; //observador de estado (L)
23 float32_t Kx_pos_f32[2] = {0.0000f, 0.0000f}; // Ganancia para control de posicion
24 float32_t Kx_vel_f32[2] = {0.0000f, 0.0000f}; // Ganancia para control de velocidad
25
26 float32_t Ki_pos_f32 = 0.0f; // Ganancia integral para control de posicion
27 float32_t Ki_vel_f32 = 0.0f; // Ganancia integral para control de velocidad

```

Listing 3.2: Fragmento de código mostrando la definición de las matrices y ganancias del sistema de control por realimentación de estados estimados (`motor.c`)

3.4. Arquitectura del Bucle de Control

El corazón de nuestro sistema reside en la función `Motor_update(void)`, contenida en el fichero `motor.c` la cual es invocada estrictamente cada 1 milisegundos.

El flujo de esta función, independientemente del algoritmo de control seleccionado (PID o Estados), sigue siempre tres:

1. **Lectura de Sensores:** Se lee el registro del hardware del *encoder* y se estima la velocidad actual mediante diferenciación numérica (para los PIDs).
2. **Lógica de Control:** Un bloque condicional (`switch` o `if/else`) deriva la ejecución hacia el algoritmo matemático que esté activo en ese momento, calculando el esfuerzo de control en Voltios.
3. **Actuación y Saturación:** La señal matemática se satura a los límites físicos del hardware (± 12 V), se escala y se aplica directamente a los registros de ciclo de trabajo (*Duty Cycle*) del generador PWM y a los pines GPIO de dirección del puente en H.

Esta modularidad nos permitirá, en capítulos posteriores, inyectar el código matemático avanzado sin alterar las rutinas de lectura de hardware ni de seguridad.

3.5. Descripción general de Motor.py

`Motor.py` es un programa que permite realizar varias tareas de monitorización y control del motor, empleando un ordenador personal. Está escrito en Python y define una única clase `MotorDashboard` en la que está encapsulado todo el código. Podemos ejecutarlo directamente en línea de comandos a través del intérprete de Python:

```
$ Python Motor.py
```

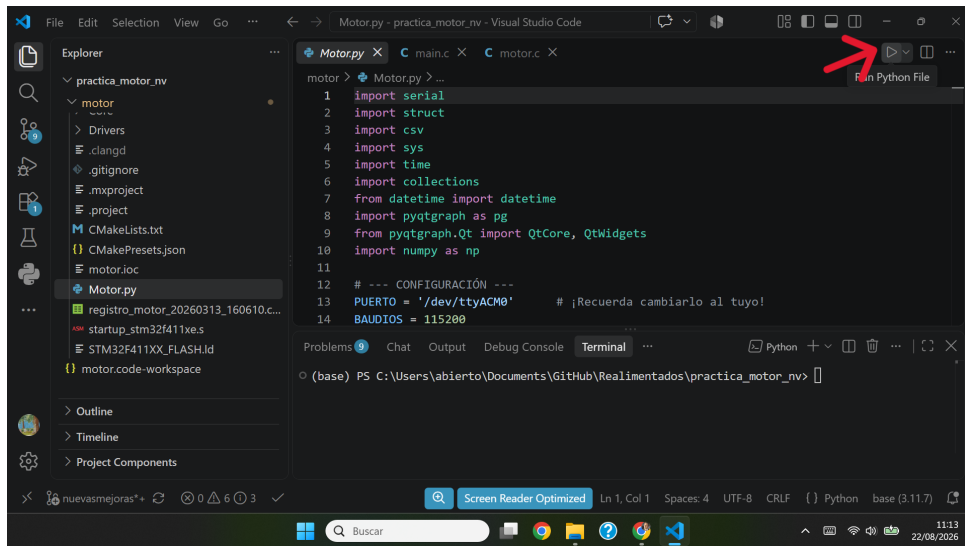


Figura 3.1: Ventana de visual Studio Code mostrando el árbol de directorios, el archivo motor.py y el botón de ejecución

También es posible ejecutarlo empleando Visual Studio Code, o cualquier otro IDE adecuado como Spyder. importante antes de ejecutarlo asegurarse de que el ordenador tiene instalado Python y los módulos de Python que importa, (ver listado 3.5).

En nuestro caso, para la práctica lo ejecutaremos desde Visual Studio Code. Si tenemos abierto el directorio `motor`, es suficiente con que seleccionemos con el ratón el archivo `motor.py` para que se abra en el editor. Una vez abierto, basta pulsar el triángulo situado en la parte superior izquierda del editor para que se ejecute. (Ver figura 3.1)

Al ejecutar el programa, éste crea una instancia (objeto) de la clase `MotorDashboard` llamado `Dashboard`. Este objeto realiza en su inicialización tres tareas principales:

Comunicación con la placa a través de un puerto serie. En primer lugar, trata de levantar las comunicaciones con la placa a través de un puerto serie. Este puerto está emulado sobre el puerto USB de la placa. Es preciso, por tanto, tener conectada la placa a un puerto USB del ordenador.

```

1 import serial
2 import struct
3 import csv
4 import sys
5 import time
6 import collections
7 from datetime import datetime
8 import pyqtgraph as pg
9 from pyqtgraph.Qt import QtCore, QtWidgets
10
11 # --- CONFIGURACION ---
12 PUERTO = '/dev/ttyACM0' # Recuerda cambiarlo por el tuyo!
13
14 BAUDIOS = 115200

```

Listing 3.3: Cabecera y primeras líneas de (`Motor.h`)

El listado 3.5, muestra las primeras líneas de código del programa. En la línea 12, se indica el puerto serie al que se ha conectado la placa. El valor que aparece por defecto `'/dev/ttyACM0'`, es el estándar para una máquina con sistema operativo linux. De todos modos, es buena practica tras conectar la placa comprobar que existe el fichero `textttttyACM0` en el directorio `texttt/dev`. Si estamos trabajando bajo Windows, deberemos comprobar a qué puerto `COM0`, `COM1`, etc se ha conectado la placa y cambiarlo en la línea 12 el valor que aparece.

En la línea 14 aparece la velocidad de comunicación en baudios. El valor que presenta, 115200 se ha ajustado para que coincida con el valor definido por el software de la placa ST. Ambos valores deben ser iguales.

Es muy importante conectar la placa al puerto USB del ordenador, antes de ejecutar el programa `Motor.py`, ya que si no encuentra el puerto abierto, el programa termina con un mensaje de error,

```

$ Error al abrir el puerto /dev/ttyACM0: [Errno 2] could not open port
/dev/ttyACM0: [Errno 2] No existe el archivo o el directorio: '/dev/ttyACM0'

```

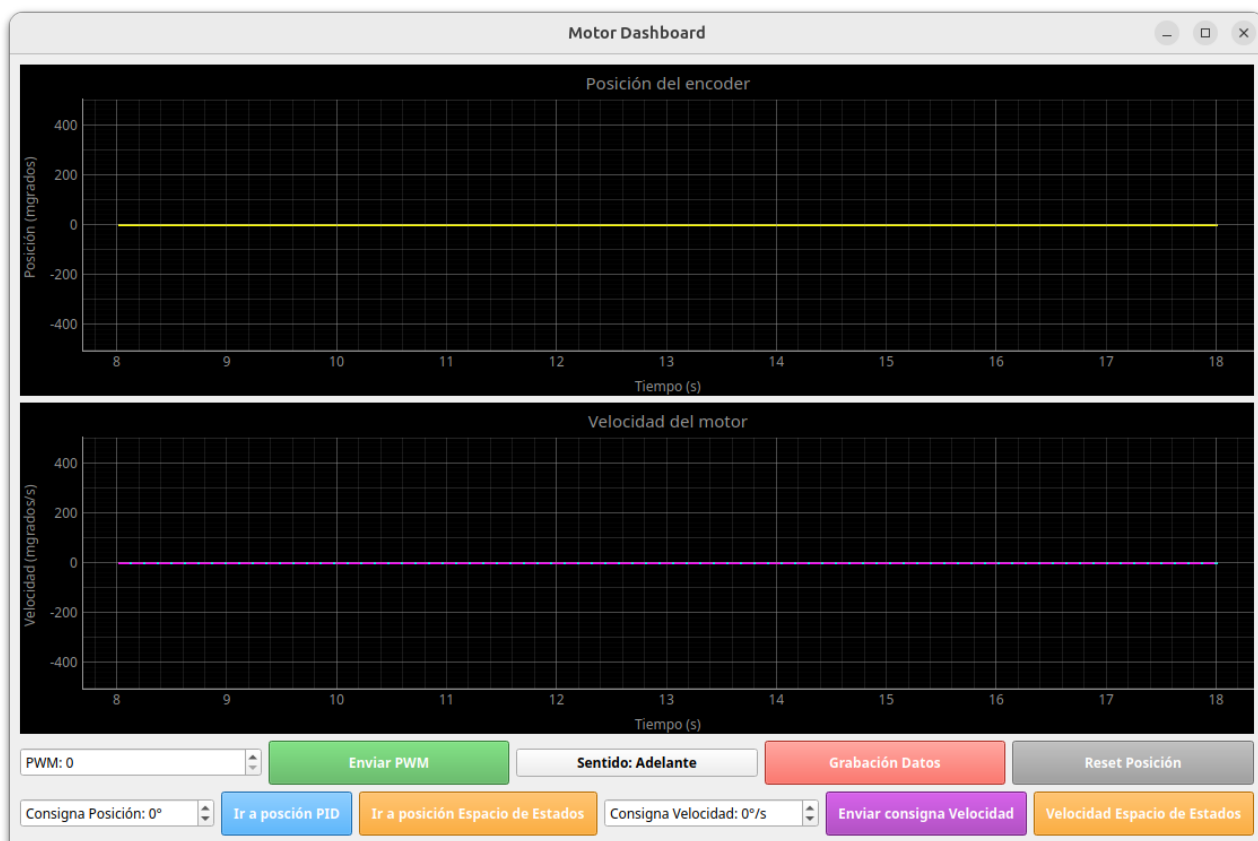


Figura 3.2: *Dashboard* Para el manejo del motor

Panel de control Una vez abierto con éxito el puerto serie, El constructor del objeto `Dashboard` abre en el escritorio una ventana que muestra un panel de control (*dashboard*) en el escritorio del ordenador. La figura 3.2 muestra una vista de dicho panel.

En la figura se pueden observar dos gráficas. La superior representa la posición angular en grados del eje del motor y la inferior la velocidad del eje del motor en vueltas por segundo, en ambos casos frente al tiempo. Se trata del tiempo en segundos medido por el reloj del microprocesador de la placa, desde que se inicia la ejecución del programa.

Inmediatamente debajo de las gráficas, aparecen dos filas de controles. La fila superior está compuesta por los siguiente elementos descritos de izquierda a derecha:

1. *spinbox PWM*. Permite introducir o seleccionar mediante las flechas arriba/abajo un valor para la señal de PWM que se transferirá a la placa del controlador de los motores. Admite un valor entero comprendido entre 0 y 999, ya que el microcontrolador, admite ese rango para generar señales de PWM. Podemos traducirlo a voltajes, sin más que tener en cuenta que es controlador del motor, trabaja a un voltaje de 12V. por tanto, un valor de PWM de 999 corresponde a 12V, uno de 500 aproximadamente a 6V, etc.
2. Botón *Enviar PWM*. Cuando se pulsa, el ordenador envía por el puerto serie a la placa Nucleo el valor de PWM indicado en la ventana del *spinbox PWM*. Ajustando por tanto el valor del voltaje suministrado al motor.
3. Botón *sentido*. Cuando se pulsa cambia el sentido de giro del motor. En la figura 3.2 El texto que aparece en el botón indica *Sentido Adelante*. Corresponde a hacer girar el eje del motor en el sentido de las agujas del reloj. Si pulsáramos el botón, cambiaría el sentido de giro y en el botón aparecería el texto. *Sentido Atrás*. el botón es un *toggle*, es decir, alterna entre los dos estados cada vez que se pulsa.
4. Botón *grabación de datos* Este botón trabaja como un *toggle*. Si se pulsa en el estado que muestra la figura 3.2 Comienza a tomar datos del movimiento del motor. Además, el texto del botón cambia a *detener y guardar*. SI se vuelve a pulsar, los datos grabados se guardan en un fichero estándar csv. Que puede abrirse desde cualquier programa de hojas de cálculo. (Ver más adelante la descripción de los datos grabados)
5. Botón *reset position*. Al pulsar el botón, el contador de pasos de encóder de la placa Nucleo se pone a cero, reseteando de este modo la posición del eje del motor.

Para la fila inferior, podemos describir los siguientes seis controles,

1. *spinbox Consigna de posición*. Permite introducir mediante el teclado o el uso de las flechas una posición angular en grados deseada a la que queremos que se estabilice la posición del eje del motor. El valor por defecto es 0° .
2. Botón *Ir a posición PID*. Este botón utiliza control PID para llevar el motor a la posición indicada en el *spinbox* anterior. No emplearemos este tipo de control en esta práctica
3. Botón *Ir a posición espacio de estados*. Este botón utiliza control en variables de estados estimados y un estimador de Luemberber para llevar el motor a la posición indicada en el *spinbox* anterior. Este será el tipo de control que emplearemos en la práctica, calculando previamente las ganancias necesarias.
4. *spinbox Consigna de velocidad*. Permite seleccionar un valor para la velocidad deseada de giro del motor de modo análogo al de los *spinboxes* anteriormente descritos. La velocidad se define en vueltas por segundo.
5. botón *Enviar consigna de velocidad*. Al pulsarlo ajusta la velocidad del motor al valor indicado en el *spinbox* anterior empleando control PID. No lo emplearemos en esta Práctica
6. botón *velocidad en espacio de estados* Al pulsarlo ajusta la velocidad del motor al valor indicado en el *spinbox* anterior empleando control en variables de estados y un estimador de Luemberger.

Una nota final: a veces, al abrir ejecutar `motor.py`, puede suceder que las gráficas se queden congeladas. Esto se debe en la mayoría de los casos a un error de sincronización del puerto serie. La solución habitual es sencillamente pulsar el botón negro de la placa, esto hace que se resetee y vuelva a funcionar normalmente.

Capítulo 4

Motor de continua en lazo abierto. Identificación del motor

En esta primera práctica nos vamos a limitar a emplear el programa de Python `Motor.py`. Para manejar el motor manualmente suministrándole un valor fijo de voltaje, y leer su posición y velocidad a través de los *encoders*.

4.1. Adquisición de datos mediante `Motor.py`

Vamos a describir en esta sección un procedimiento estándar para adquirir y guardar datos del movimiento del motor. El procedimiento se va a describir empleando valores fijos de PWM, pero es directamente aplicable a cualquier otro uso del motor.

En primer lugar, debemos conectar los cables de alimentación de la placa de control a la fuente de 12V y la salida USB a un puerto USB del ordenador.

Ejecutamos el script de Python `Motor.py`. Si la placa ha sido correctamente programada y el puerto serie está funcionando correctamente, veremos en la gráfica de posición y velocidad unas líneas horizontales en cero y como se desplazan los valores de tiempo en el eje x.

Es aconsejable, en primer lugar comprobar que la comunicación entre la placa y el ordenador funciona correctamente. Para ello introducimos un valor en el *spinbox* PWM, por ejemplo 500, que equivaldría a suministrar 6 voltios al motor (Recordad que tenemos 1000 divisiones para el ancho del pulso (duty cycle) de la señal de PWM. Por tanto, podemos seleccionar 1000 valores de voltaje entre 0V y 12V). Pulsamos el botón enviar PWM y deberíamos ver que el motor se pone en marcha y que en las gráficas del panel de control se ve una recta creciente para la posición y una señal con una fuerte oscilación pero de media constante para la velocidad.

Si todo ha salido según lo previsto, pulsamos el botón *reset posición* y el motor se parará con la posición fija en el valor cero¹.

Pulsamos el botón grabación de datos y a continuación volvemos a pulsar el botón enviar PWM. Esperamos el tiempo que deseemos y pulsamos de nuevo este último botón, en el que ahora figurará el texto detener y guardar.

Inmediatamente se creará un archivo, con los datos recogidos, cuyo nombre contiene la fecha y hora en que se creó: `registro_motor_aaaammdd_hhmmss.csv`.

La tabla 4.1 muestra un ejemplo de la estructura del archivo generado. En concreto el nombre de este archivo es `registro_motor_20260313_152318.csv`.

Tiempo (s)	PWM (con signo)	Posición (grados)	Velocidad (grados/s)
1588.71	250	1	0.0
1588.72	250	1	0.0
1588.73	250	1	0.0
1588.74	250	1	0.0
1588.75	250	1	0.0
...	...	...	...

Tabla 4.1: Estructura del registro de datos del motor (muestra de las primeras filas).

Es el archivo puede manejarse desde cualquier hoja de cálculo o importarlo en Python para operar con los datos obtenidos.

¹En algún caso, debido a la inercia del motor, puede ser necesario pulsar el botón un par de veces.

Es interesante hacer notar cómo el tiempo incrementa en valores regulares de 10ms, tal y como se programó en `main.c`.

Practica 1

1. Identificar todos los componentes de la planta. Placa de desarrollo, Placa de expansión, Motor, Fuente de alimentación, etc.
2. Comprobar que todo el *Hardware* está configurado de acuerdo a la descripción de estas notas.
3. Programar y descargar el software de control de la placa. Comprobar que funciona correctamente y que se comunica con el programa `Motor.py`.
4. Probar distintos valores de PWM y comprobar que funciona correctamente. Comprobar que se puede cambiar el sentido de giro sobre la marcha. Determinar la zona muerta del motor (voltaje mínimo por debajo del cual no arranca). Si se quiere, se pueden probar los botones de control del motor, aunque no harán nada, ya que todas las ganancias de los controladores están puestas a cero por defecto.
5. Recoger datos para distintos valores del voltaje de entrada (tomar por ejemplo 10 valores equiespaciados desde el final de la zona muerta hasta los 12V) es suficiente con tomar datos durante 3 ó 4 segundos. Guardar los archivos obtenidos que serán utilizados en prácticas posteriores.

4.2. Modelo e identificación del motor

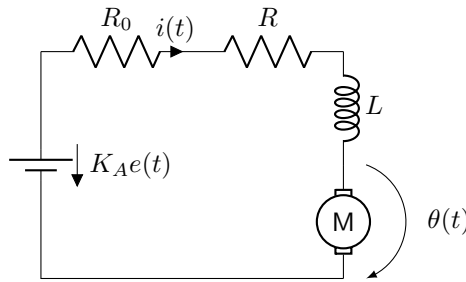


Figura 4.1: Circuito equivalente para un motor de continua

Modelo del motor El circuito de la figura 4.1 representa un motor de corriente continua y su sistema de alimentación. Podemos considerar el sistema compuesto de dos partes:

- Una parte eléctrica formada por un variador (Punto en H más señal de PWM) que recibe un voltaje de entrada $e(t)$ y devuelve un voltaje amplificado $K_A e(t)$. Además el amplificador tiene una resistencia interna R_0 .

La parte eléctrica del motor (bobinado) viene representada por una autoinductancia L y una resistencia R . Por último, podemos considerar que la caída de tensión (fuerza contra electromotriz) debida al giro del motor es proporcional a la velocidad angular $\dot{\theta}$ con la que gira el rotor. La constante de proporcionalidad a depende de las características del motor. Si sumamos todas las caídas de tensión en el circuito:

$$L \frac{di(t)}{dt} + (R + R_0)i(t) + a \frac{d\theta(t)}{dt} = K_A e(t) \quad (4.1)$$

- Una parte mecánica que podemos asociar al momento de inercia J del rotor y a la resistencia al giro ofrecida por el rozamiento —que consideraremos proporcional a la velocidad angular a través de un coeficiente de rozamiento viscoso μ .

El par ejercido por el motor, es proporcional a la intensidad que circula por el bobinado. La constante de proporcionalidad vuelve a ser la constante del motor a . Por último, podemos suponer que la carga que mueve el motor ejerce un par T_L :

$$J \frac{d^2\theta(t)}{dt^2} + \mu \frac{d\theta(t)}{dt} = ai(t) - T_L(t) \quad (4.2)$$

Sin embargo, la dinámica debida a la autoinducción L es mucho más rápida que la dinámica propia de la mecánica del motor. Esto nos permite simplificar el modelo eliminando la variación de la intensidad –es tan rápida que el resto del sistema siempre la *ve* en su estado estacionario–.

$$L \frac{di(t)}{dt} = 0 \Rightarrow (R + R_0)i(t) + a \frac{d\theta(t)}{dt} = K_A e(t) \quad (4.3)$$

Si despejamos la intensidad de la ecuación 4.3 y sustituimos en 4.2, obtenemos:

$$J \frac{d^2\theta(t)}{dt^2} + \left(\mu + \frac{a^2}{R + R_0} \right) \frac{d\theta(t)}{dt} = \frac{aK_A}{(R + R_0)} e(t) - T_L(t) \quad (4.4)$$

La ecuación anterior la podríamos reescribir como un modelo en representación en variables de estados con dos estados θ y $\omega = \dot{\theta}$

$$\dot{\theta}(t) = \omega(t) \quad (4.5)$$

$$\dot{\omega}(t) = -\frac{1}{J} \left(\mu + \frac{a^2}{R + R_0} \right) \omega(t) + \frac{1}{J} \frac{aK_A}{(R + R_0)} e(t) - \frac{1}{J} T_L(t) \quad (4.6)$$

$$(4.7)$$

Alternativamente, si obtenemos la transformada de Laplace de la ecuación 4.4, tendríamos dos funciones de transferencia una para la relación tensión-posición y otra para la relación par de carga-posición:

$$\theta(s) = \frac{1}{s} \frac{\frac{1}{J} \frac{aK_A}{R+R_0}}{s + \left(\mu + \frac{a^2}{R+R_0} \right) \frac{1}{J}} e(s) - \frac{1}{s} \frac{\frac{1}{J}}{s + \left(\mu + \frac{a^2}{R+R_0} \right) \frac{1}{J}} T_L(s) \quad (4.8)$$

En la mayoría de los casos desconocemos los valores de los parámetros del motor. Sin embargo, los agrupamos en tres simples constantes, que podemos estimar experimentalmente:

$$k_e = \frac{1}{J} \frac{aK_A}{R + R_0}, \quad p = \left(\mu + \frac{a^2}{R + R_0} \right) \frac{1}{J}, \quad k_m = \frac{1}{J} \quad (4.9)$$

Si, además consideramos un par de carga nulo $T_L(t) = 0, \forall t$, los modelos se simplifican pasando a ser modelos SISO:

$$\begin{bmatrix} \dot{\theta} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & -p \end{bmatrix} \cdot \begin{bmatrix} \theta \\ \omega \end{bmatrix} + \begin{bmatrix} 0 \\ k_e \end{bmatrix} e(t) \quad (4.10)$$

$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} \theta \\ \omega \end{bmatrix} \quad (4.11)$$

$$\theta(s) = \frac{k_e}{s(s+p)} e(s) \quad (4.12)$$

Identificación del motor Si calculamos la respuesta de nuestro sistema a una entrada escalón de valor $e(t) = V$,

$$\theta(t) = V \frac{k_e}{p^2} e^{-pt} + V \frac{k_e}{p} t - V \frac{k_e}{p^2} \quad (4.13)$$

La respuesta angular en estado estacionario es una línea recta, que indica que el motor, tras pasar la fase transitoria, gira con velocidad constante. La figura 4.2 muestra un ejemplo de la respuesta de un motor como el que estamos modelando. Si obtenemos la pendiente de la recta correspondiente a la solución estacionaria y su corte con el eje y , podemos estimar los parámetros K_e y p de nuestro motor. Este proceso puede hacerse a partir de datos experimentales del motor, en cuyo caso recibe el nombre de proceso de identificación del motor.

Práctica 2

1. Obtener a partir de cálculo simbólico la expresión correspondiente a la posición angular de un motor de continua (ecuación 4.13). Obtenerla a partir del modelo en variables de estado. Calcular también la expresión correspondiente a la velocidad angular.

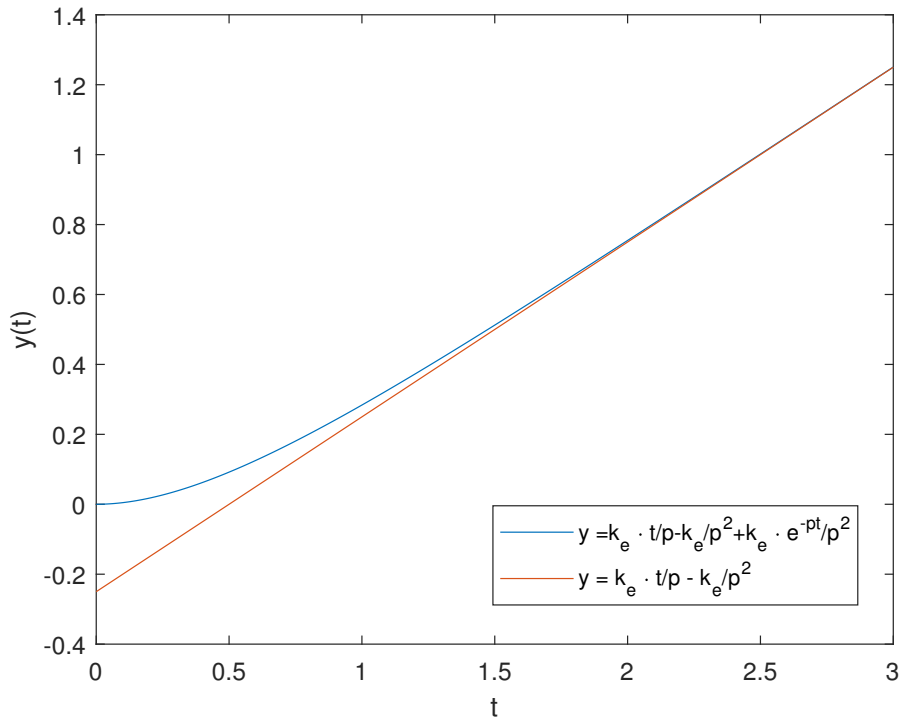


Figura 4.2: Respuesta de un motor de cc a un voltaje de entrada escalón

2. Construir un modelo del motor de cc en Python o Matlab. Se puede hacer a partir de las ecuaciones de estado, pero de modo que se pueda obtener tanto la posición como la velocidad angular. El modelo deberá tomar como variables los parámetros k_e y p . Emplear los datos obtenidos en la práctica anterior, en lazo abierto para identificar el motor real. Es decir para obtener sus parámetros k_e y p . Para ello:
 - Calcular, a partir de los datos de posición y tiempo, por regresión lineal la pendiente y el término independiente de la respuesta en estado estacionario. Es necesario considerar los intervalos de tiempo transcurridos desde que arrancó el motor. Es decir tomar como instante cero, el correspondiente al primer valor de la señal de PWM distinto de cero.
 - Estimar los valores de K_e y p para cada valor del voltaje de entrada. Representar K_e frente al voltaje y p frente al voltaje ¿Qué se puede concluir?
3. Introducir los valores obtenidos de K_e y p en el modelo del motor elaborado en apartado 2 (obtener unos valores promedio razonables) Comparar los resultados del modelo con el sistema real.

Capítulo 5

Control del motor por realimentación de estados estimados

En esta sección vamos a construir un regulador completo para controlar la posición del motor. Empezaremos construyendo un modelo realista completo del sistema, a partir de los datos obtenidos en el proceso de identificación del capítulo anterior.

5.1. Un modelo completo del motor

Modelo ideal del motor Vamos a construir un modelo del motor que emplee exactamente las mismas señales de entrada y salida que emplea el motor real. Además, tendremos en cuenta algunas de las no linealidades más características: la existencia de un voltaje máximo que no se puede superar y la existencia de un voltaje mínimo por debajo del cual el motor no funciona.

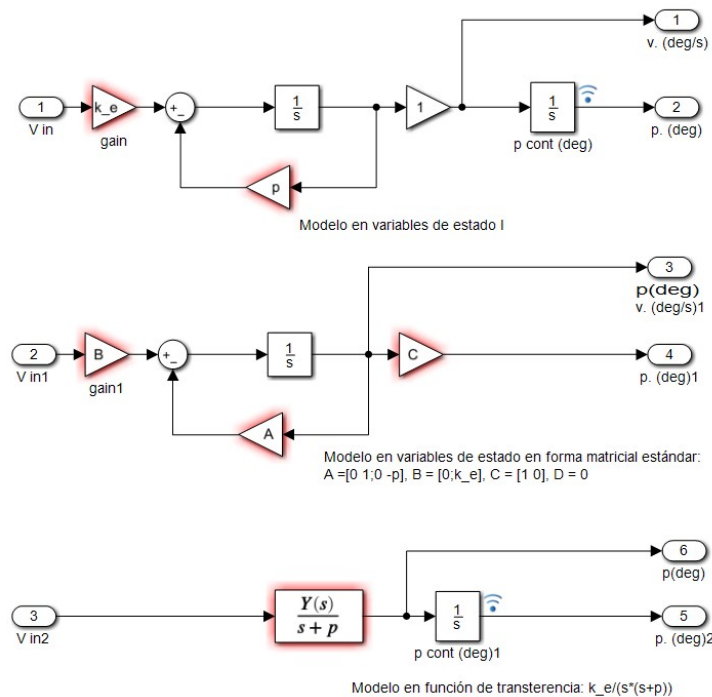


Figura 5.1: Esquemas de Simulink para el modelo del motor ideal

La figura 5.1 muestra tres versiones del modelo sencillo descrito en la sección anterior para un motor ideal de corriente continua. Las dos primeras son versiones en variables de estados y la tercera una versión empleando la función de transferencia. Los tres modelos son equivalentes y cualquiera de ellos se puede usar como núcleo de un modelo más realista del motor que estamos empleando. El modelo basado en la función de transferencia emplea en realidad dos en cascada. La razón es tener acceso no solo a la posición –que es la salida del sistema– sino también a la velocidad que corresponde a la salida de la primera función de transferencia. Para ajustar nuestro modelo al motor real empleado en la práctica, deberemos ajustar los valores de K_e y p a los valores obtenidos

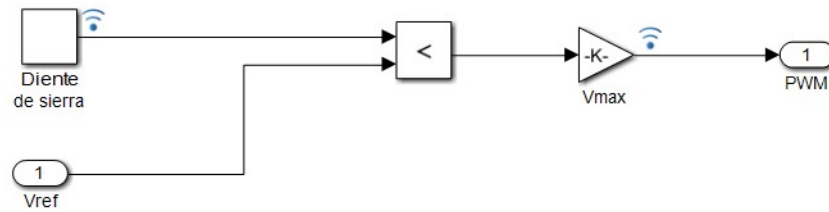


Figura 5.2: Esquema de Simulink del bloque construido para generar una señal de PWM

en el proceso de identificación del motor, descrito en el apartado anterior. Se ha añadido una ganancia a la salida del modelo que podría ajustarse para modelar la reductora. Se trata tan solo de una constante y dado que hemos identificado el motor empleando la posición del eje de la reductora, podemos asignarle directamente valor 1.

Modelo del sistema de alimentación Para aproximar nuestro modelo al motor de la práctica, el siguiente paso es construir un modelo de su sistema de alimentación. Sabemos que el motor recibe un nivel de tensión variable mediante el empleo de una señal de PWM. Para emular dicho sistema, creamos un nuevo modelo de Simulink como el que se muestra en la figura 5.2. El modelo se construye a partir de un bloque que reproduce una señal en diente de sierra, construido a partir de un bloque *Repeating Sequence* de la librería Simulink/Sources. Este bloque espera como parámetros un vector de instantes de tiempo (*Time values*) comprendidos entre 0 y un tiempo final T , y un vector de valores de salida (*Output values*) de la misma longitud que el vector de tiempos. El módulo, durante su ejecución, da como salida la secuencia introducida repetida periódicamente cada T segundos. En nuestro caso empleamos los siguientes vectores como parámetros:

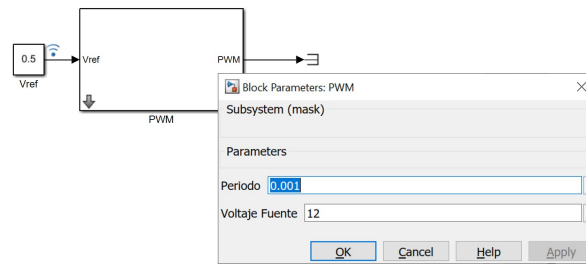
Time Values: `linspace(0,T,100)`
Output values: `linspace(0,Vmax,100)`

El resultado de la señal de salida del bloque será una sucesión de valores crecientes entre 0 y V_{max} , que se repiten periódicamente cada T segundos. La señal toma por tanto la forma de un diente de sierra. El bloque comparador compara el valor de entrada, V_{ref} , en cada instante con el valor de el diente de sierra. V_{ref} representa el valor de tensión continua que queremos emular con la señal de PWM. Mientras el valor del diente de sierra sea menor que el de la entrada V_{ref} , la comparación devuelve un 1, en caso contrario devolverá un 0. La ganancia multiplica el 1 o el 0, por el valor V_{max} . El resultado es que a la salida tendremos un valor igual a V_{max} durante un periodo igual al que la entrada es mayor que el valor del diente de sierra, y a partir de ahí y hasta el final de periodo T , valdrá cero. Es fácil comprobar que la señal así construida corresponde a una señal de PWM de valor V_{max} y periodo T . El tiempo que la señal está activada (*duty cycle*) corresponde precisamente al tiempo en que la entrada supera en valor a la señal en diente de sierra. Así por ejemplo, si elegimos $V_{ref} = V_{max}/2$ la señal valdrá V_{max} durante un tiempo $t = V_{max}/2$. Podemos enmascarar todo el modelo de la figura 5.2 en único bloque y definir como parámetros V_{max} y T .

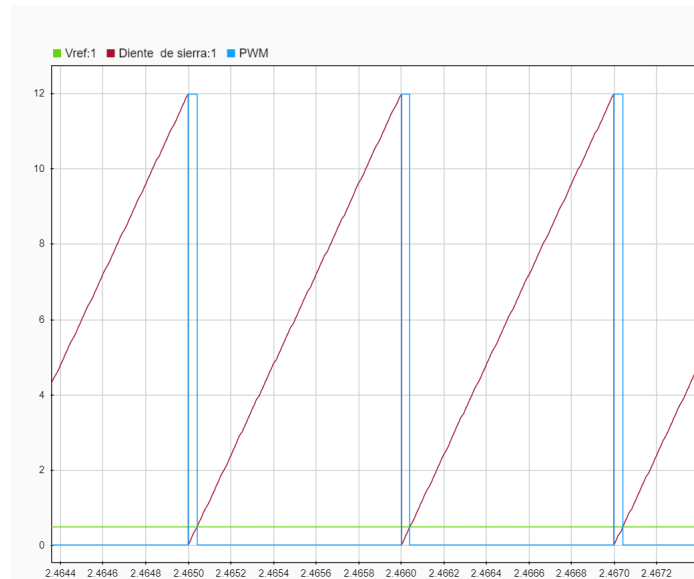
Un detalle importante a la hora de manejar el bloque de PWM descrito tiene que ver con el modo en que Simulink maneja los tiempos de los modelos. Por defecto, Simulink emplea integradores de tiempo variable, de modo que el tiempo entre pasos de integración se ajusta a la convergencia de las soluciones del modelo. En nuestro caso, podría suceder que los pasos empleados por Simulink sean tan grandes que nos recorte la señal de PWM. Para evitar esto es importante ajustar el paso máximo de integración, en el modelo en que se está empleando el bloque PWM, a la décima parte (o menor) que el periodo T asignado a la señal de PWM. El ajuste se hace en la ventana *Parameter configuration, Max step size*. Un valor razonable para un $T = 0,001s$ sería *Max step size* = $1e - 5$. Por último podemos probar el funcionamiento del bloque para varios valores de entrada. Las figuras 5.3a y 5.3b muestran el nuevo bloque construido y su salida comparada con la señal en diente de sierra, y el voltaje de referencia, para valores $V_{max} = 12$ y $T = 0,001$ y un valor $V_{ref} = 0,5$.

Es interesante hacer notar que el bloque construido, –al igual que pasa con el controlador real– no permite alimentar el motor con un voltaje mayor que el voltaje de la fuente de alimentación. Además, al dar solo valores positivos, solo permitiría mover el modelo del motor en un sentido de giro.

Nos faltaría emular el efecto del puente en H. Para ello, añadimos al modelo construido los siguientes bloques de la librería *Simulink/Math operations*: Un bloque *Abs*, que obtenga el valor absoluto de la señal V_{ref} . Un bloque *sign*, que extraiga el signo de la señal V_{ref} . Un bloque *Prod* que multiplique la salida del bloque PWM por el signo de la señal V_{ref} . El modelo resultante se muestra en la figura 5.4. Es fácil comprobar que si introducimos una señal V_{ref} negativa, la señal de PWM a la salida, será negativa, emulando así el efecto del cambio de entradas de un puente en H.



(a) Bloque desarrollado y ventana de parámetros



(b) Señales empleadas en el generador de PWM

Figura 5.3: Bloque para simular una señal de PWM

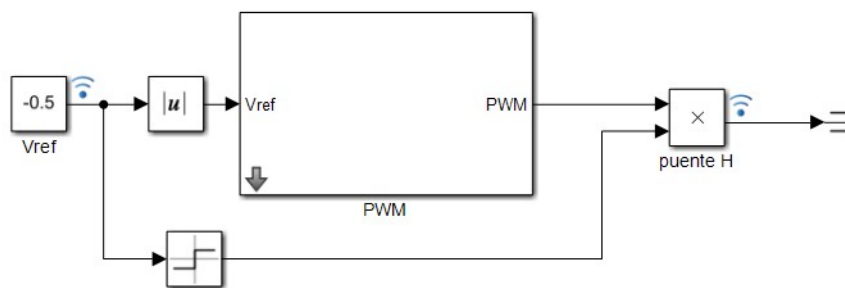


Figura 5.4: Modelo del alimentador de tensión al motor incluyendo el cambio de signo de voltaje.

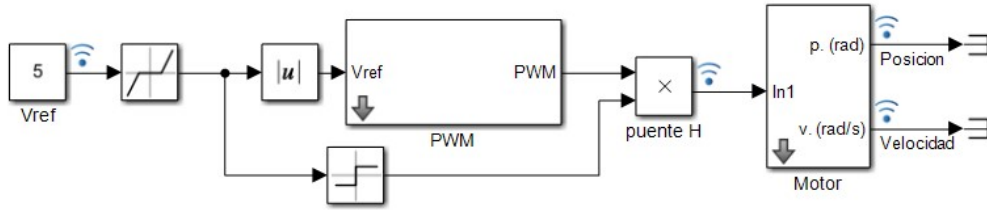


Figura 5.5: Esquema del modelo incluyendo el motor y su sistema de alimentación

Una vez construido el sistema de alimentación del motor, podemos crear un modelo que incluya tanto la alimentación como el modelo de motor ideal, empleando cualquiera de los esquemas de Simulink de la figura 5.1. La figura 5.5 muestra un ejemplo. En este caso, se ha optado por emplear el primero de los modelos en variables de estado de la figura 5.1. El modelo del motor se ha enmascarado definiendo como parámetros del modelo las constantes p y Ke . Se han conservado como variables de salida la posición y velocidad (angulares) del eje del motor. Aunque dichas variables se han etiquetado como $p(rad)$ y $v(rad/s)$ las unidades dependen en realidad del valor que se asigne a las constantes del modelo. Se ha añadido a la entrada del modelo un bloque *dead zone* de la librería *Simulink/discontinuities*. Este bloque sirve para simular la zona muerta del motor; es decir, el voltaje mínimo por debajo del cual el motor no se moverá. Estrictamente debería ir en el bloque que simula el motor, pero esto complica el modelo. En realidad, la señal de PWM siempre da al motor el voltaje máximo, lo que realmente limita es la corta duración del *duty cycle* cuando el voltaje de referencia es muy bajo. Modelarlo exactamente ralentiza enormemente el tiempo de integración y genera errores de precisión. La forma en que se ha modelado dará valores razonables, siempre que el periodo de la señal de PWM sea razonablemente corto.

Modelado de los encoders Una vez modelados el motor y su sistema de alimentación, debemos modelar también el comportamiento de los *encoders*. Como se ha descrito en las secciones anteriores, el motor real dispone de dos *encoders* en cuadratura que suministran un pulso –flanco de subida y bajada– por cada grado que avanza el eje del motor. Podemos diseñar un modelo para ellos que utilice como entrada la salida del modelo ideal del motor. Como los *encoders* están situados en cuadratura, la distancia recorrida por el motor entre un flanco de subida y bajada de un *encoder* es de 2 grados. Además las lecturas están entrelazadas. Entre flancos, cada *encoder* deberá dar un valor constante — 1 ó 0 — dependiendo de la posición del eje del motor. La figura 5.6 ofrece un posible modelo para los *encoders*.

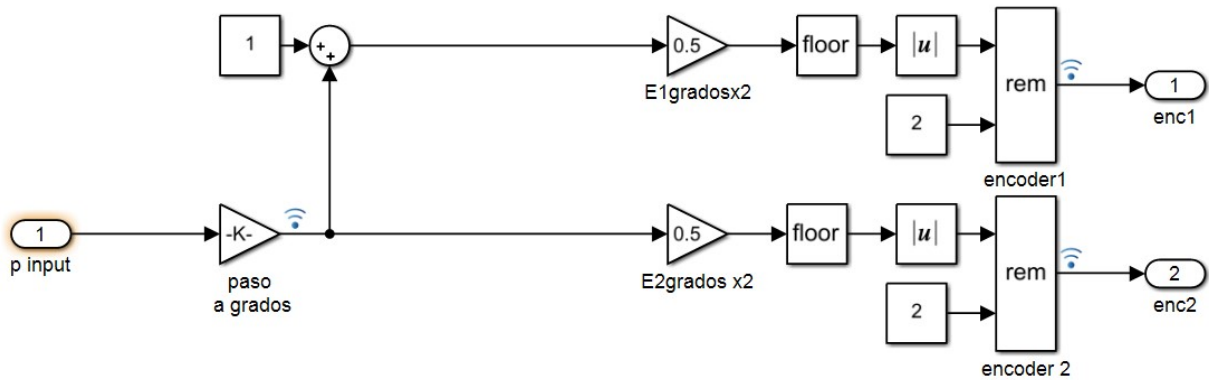
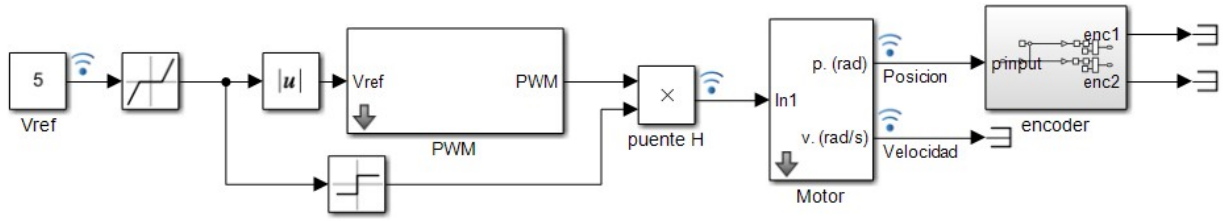


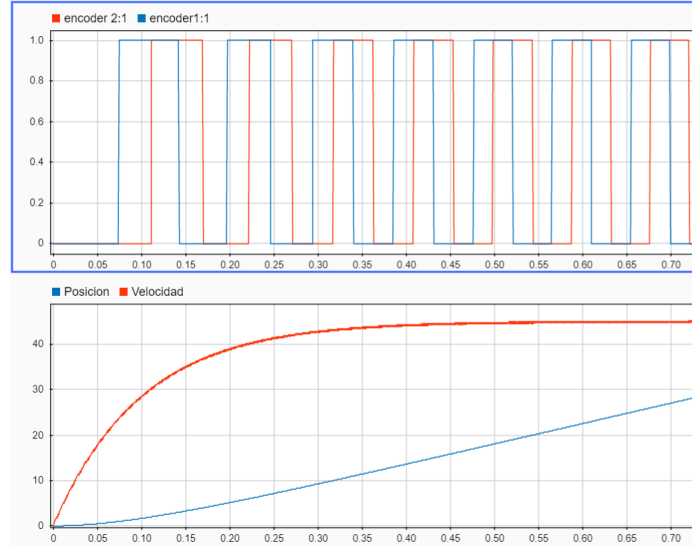
Figura 5.6: Modelo de Simulink para dos *encoders* en cuadratura

El modelo toma como entrada la salida del modelo del motor ideal, correspondiente a la posición angular del eje. La ganancia etiquetada como *paso a grados* solo se emplea si la salida del motor se mide en radianes, si se mide en grados tomará valor 1. Se han introducido dos bloques no empleados anteriormente, pertenecientes a la librería *Simulink/math operations*. Se trata del bloque *floor* que redondea un número hacia cero y el bloque *rem* que calcula el resto de la división entera de la entrada superior entre la entrada inferior.

Para analizar su funcionamiento, consideremos el motor parado al inicio de una simulación. El valor de la posición p input del eje del motor es 0. El camino que lleva a la salida *enc1* corresponderá al *encoder A*. El modelo suma un 1 constante a dicho valor y divide el resultado por dos. Tras el redondeo hacia cero, y el cálculo del



(a) Modelo del motor incluyendo el bloque del modelo de los encoders



(b) Modelo del motor incluyendo el bloque del modelo de los encoders

Figura 5.7: Modelo completo del motor

resto de la división entre dos, el valor de la salida será 0. La salida *enc2*, correspondiente al *encoder* B, también tomará valor cero. Supongamos que avanza un grado la posición del eje del motor, la lectura del *encoder* A cambiará a 1, mientras que la del *encoder* B seguirá siendo cero. Si el motor avanza un grado más, la lectura de A se mantiene en su valor 1, y la de B pasará a valer 1. Es fácil observar que, en el periodo comprendido entre incrementos de un grado, ambos *encoders* mantendrán constante su valor de salida y que la secuencia temporal de ambas señales reproduce la que cabría esperar para dos *encoders* en cuadratura. Para probarlo, podemos encapsular nuestro modelo en un bloque, conectar su entrada a la salida del modelo del motor y simular el modelo completo. Las figuras 5.7a y 5.7b muestran el modelo completo y las señales de salida obtenidas para los *encoders*.

Con todos los elementos anteriores hemos construido un modelo completo del motor. Podemos alimentarlo con un voltaje de referencia y obtener los valores que nos darían las lecturas de sus encoders. Por último podríamos emplear el mismo bloque *lector encoders* descrito en la figura ?? para leer los pulsos del bloque *encoder* y obtener los valores de la posición angular y la velocidad del eje de la reductora del motor. Podemos además comparar dichas salidas con las que suministra el bloque *motor* que hemos construido y ver el efecto de la cuantización y el muestreo sobre las salidas del motor. La figura 5.8 muestra un ejemplo en que se comparan dichas salidas. Se ha buscado un caso en el que se notan especialmente estos efectos. Así, los escalones en la lectura de la posición realizada con los *encoder* son claramente visible en la parte superior de la figura. Además se ha elegido un tiempo $\Delta t = 1s$ en que se observa claramente el efecto del retenedor de orden cero sobre la medida de la velocidad. El algoritmo empleado para estimar la medida de la velocidad es sencillo, pero no es nada robusto. El motor alcanza su velocidad estacionaria en menos de 1s pero el lector no es capaz de detectarlo. Una vez alcanzada dicha velocidad, el lector oscila por encima y por debajo del valor real. Si disminuimos el valor de δT , esta oscilación se hará aún más acusada. Si lo aumentamos, acumularemos aún un retardo mayor en la detección de los cambios de velocidad. No podemos por tanto emplear esta medida directamente para controlar el motor.

Práctica 3

1. Construir un modelo completo del motor, a partir del modelo de motor ideal construido en la Práctica 2. Añadir el mismo bloque de lectura de *encoders* construido para leer los *encoders* del motor real.

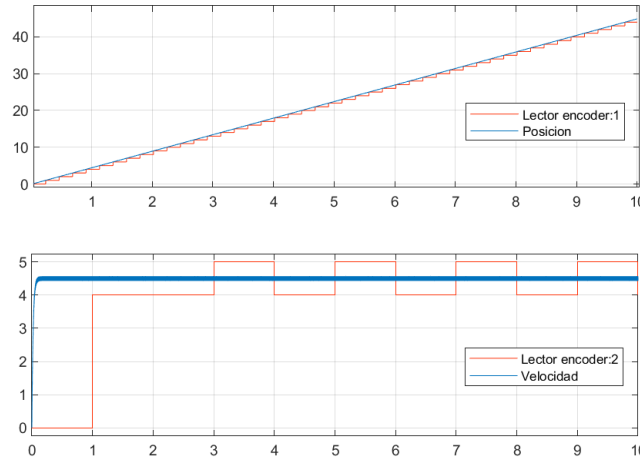


Figura 5.8: Comparación entre las variables de salida posición y velocidad obtenidas por el modelo del motor y el valor de dichas variables obtenidas a partir de la lectura de los *encoders*

2. Simular el comportamiento del modelo para distintos valores del voltaje de entrada V_{ref} .

- Comparar la salida directa (posición y velocidad) del motor con la que se obtiene a partir de la lectura de los *encoders*
- Comparar los valores de las lecturas de velocidad y posición obtenidas a través de los *encoders* para el motor real y para el modelo.

5.2. Diseño de un controlador de la posición del motor por realimentación de estados estimados

5.2.1. Controlador para el motor ideal

Para estudiar el diseño del controlador, empezaremos considerando el motor ideal en variables de estado descrito por las ecuaciones 4.10, 4.11. El objetivo final es desarrollar un controlador de posición angular del eje del motor. Si trabajamos sobre el modelo del motor ideal, podemos empezar analizando su estabilidad. Si calculamos los autovalores de la matriz del sistema (4.10),

$$A = \begin{bmatrix} 0 & 1 \\ 0 & -p \end{bmatrix}, |\lambda I - A| = 0 \rightarrow \lambda_1 = 0, \lambda_2 = -p \quad (5.1)$$

El sistema tiene un polo real negativo y un polo en el origen. Lo que corresponde al comportamiento físico observable. Si suministramos una entrada escalón ($V_{ref} = cte$) el motor se mueve indefinidamente con velocidad constante.

Realimentación de estados. Podemos empezar por diseñar un controlador por realimentación de estados, para mover los polos del sistema. Podemos emplear para calcular las ganancias de la realimentación cualquiera de las funciones definidas en Matlab para este propósito: **acker** o **place**. Aunque teóricamente los polos pueden situarse en cualquier lugar del semiplano negativo complejo, hay que tener en cuenta que cuanto más a la izquierda los situemos más grande será el valor de la variable (V_{ref}) demandada por el motor durante el transitorio. Por otro lado, si elegimos polos cercanos al eje imaginario, la respuesta del sistema puede ser muy lenta y demandar tensiones tan bajas que el motor real entre en su zona muerta y no alcance el estado estacionario. Un posible ajuste inicial, puede ser, situar ambos polos relativamente cerca del valor p obtenido para el polo p del motor durante la identificación realizada en las prácticas anteriores. Por ejemplo podemos calcular cuáles serían las ganancias $k = [k_1, k_2]$ de la realimentación de estados si situamos los polos en $p_1 = 0,5p$ y $p_2 = 0,5p$ ¹. Podemos además obtener la evolución de los estados si partimos del modelo de motor ideal que diseñamos en la práctica anterior.

La figura 5.9b muestra los resultados de un ejemplo de dicho sistema. Se ha considerado la posición inicial del motor $x_1 = 2^\circ$ y la velocidad inicial $x_2 = 1^\circ/s$. Los valores $K_e = 100$ y $p = 50$ se han tomado arbitrariamente y no tienen por qué corresponder con los del motor estimado (De hecho K_e para el motor real es un orden de

¹Si colocamos ambos polos en la misma posición será preciso emplear **acker** para obtener las ganancias

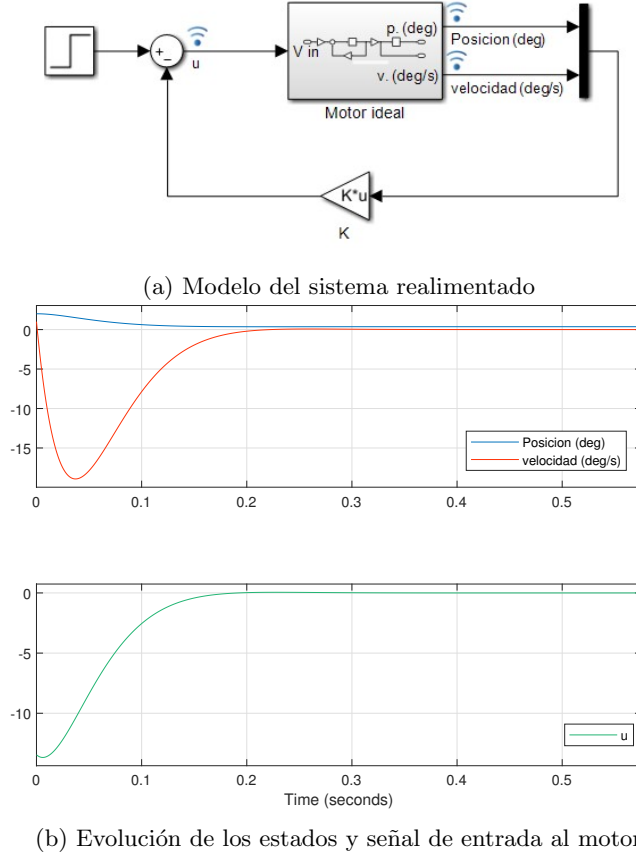


Figura 5.9: Realimentación de estados para el motor ideal

magnitud mayor). La entrada con el bloque escalón se ha puesto a cero. La señal u representaría en este modelo el voltaje V_{ref} suministrado al motor. La realimentación de estados diseñada es capaz de llevar el motor a su posición cero, como era de esperar. Dado que hemos fijado la entrada a cero, la realimentación de estados está respondiendo exclusivamente a los valores iniciales de los estados del sistema.

Diseño de un estimador Como ya hemos descrito más arriba, de las dos variables de estado que definen nuestro sistema, la velocidad resulta difícil de precisar, sobre todo cuando el motor acelera o frena. Se hace por tanto necesario diseñar un sistema que emplee realimentación de estados estimados. Para diseñarlo, gracias al principio de separación, podemos calcular la ganancia L del estimador, empleando de nuevo **acker** o **place** y conservar las ganancias K obtenidas para la realimentación de estados. Podemos colocar los polos del estimador a la izquierda del polo p del motor, por ejemplo $p_{e1} = 1,2p$ y $p_{e2} = 1,2p$. Podemos ahora repetir la simulación, empleando la realimentación de los estados estimados en lugar de los estados reales. Para ello, añadimos al modelo de Simulink de la figura 5.9a un estimador de estados, empleando de nuevo los datos obtenidos para el motor real en la práctica de identificación. El modelo completo resultante se muestra en la figura 5.10a. La estructura del estimador, corresponde al modelo completo en variables de estado. Las matrices A , B , C corresponden a las matrices del sistema (motor ideal), $K = [K_1, K_2]$, corresponden al vector de ganancias de la realimentación de estados, obtenidas anteriormente, y $L = [L_1, L_2]^T$ es el vector de ganancias del estimador. Es importante hacer notar que los bloques de ganancias del modelo de Simulink hay que configurarlos para que calculen productos matriciales. El modelo está construido de modo que toma todos los parámetros del sistema del *work space* de Matlab. Al modelo se le ha añadido una ganancia de entrada a la que se ha dado valor 1. Corresponde al valor de la ganancia F_c de acción directa. El valor asignado es arbitrario por lo que no cabe esperar que el motor alcance la posición de consigna indicada por la entrada escalón². La figura 5.10b muestra la evolución de los estados del sistema así como la de los estados estimados, para una entrada escalón de valor 5. Puede observarse como el motor alcanza una posición final, y como los estados estimados convergen a los reales antes de alcanzar el estado final. La convergencia es lo suficiente rápida para que el sistema siga una trayectoria

²Para que la acción directa (*feedforward*) llevara el motor a la posición indicada por el escalón de entrada, $y = x_r$, Es preciso ajustar el valor de F_c ,

$$F_c = (C(BK - A)^{-1}B)^{-1} \quad (5.2)$$

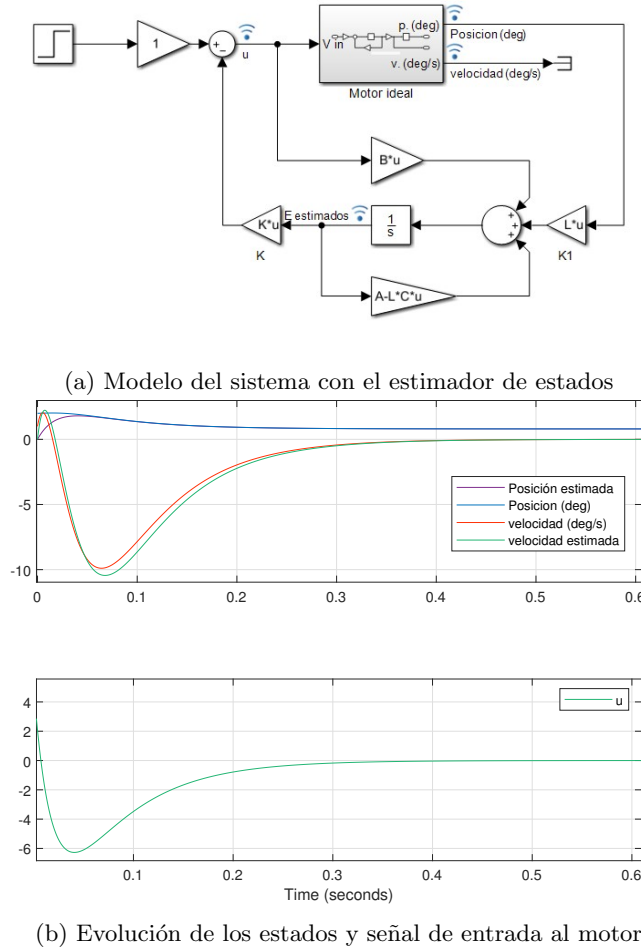


Figura 5.10: Realimentación de estados estimados para el motor ideal. La entrada es un escalón de valor 5

muy parecida a la del caso anterior, 5.9b en que se empleaba realimentación de estados reales. Lógicamente, se podría acelerar dicha convergencia, desplazando los polos del estimador más a la izquierda. Se observa también que el sistema no alcanza el valor de la entrada. Esto era de esperar puesto que no hemos ajustado el valor de la ganancia F , para que esto ocurra. De hecho, no vamos a calcular dicho ajuste. En su lugar, añadiremos directamente acción integral a nuestro sistema de control.

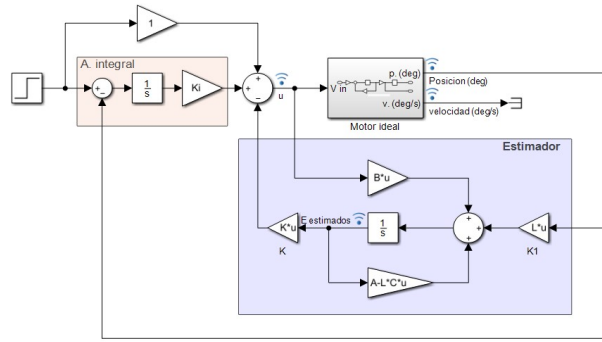
Acción integral Si añadimos acción integral al sistema de control, conseguimos que el motor siga a la entrada de referencia. Debemos para ello calcular la ganancia k_i correspondiente a la acción integral. Tenemos dos modos de hacerlo. Podemos colocar de nuevo los tres polos en posiciones pre-establecidas y emplear **acker** o **place** para obtener las ganancias $K = [K_1, K_2, K_i]$. Esto nos da la oportunidad de mantener los polos de la realimentación de estados en la misma posición que estaban cuando no se empleaba acción integral. Tenemos también la alternativa de conservar las ganancias calculadas previamente para la realimentación de estados y emplear la técnica del lugar de las raíces para obtener la ganancia k_i . Con esto tendríamos nuestro motor ideal controlado en posición. La figura 5.11a muestra el controlador completo. Se ha recuadrado en rojo la parte que corresponde a la acción integral y en azul la parte que corresponde al estimador de estados.

Para este ejemplo, se ha cambiado la posición de los polos. Así los polos del motor se han situado en $p_1 = -15, p_2 = -15$, el correspondiente al estado de la acción integral en $p_i = -20$ y los polos del observador en $p_{e,1} = -30, p_{e,2} = -30$. El modelo del motor sigue empleando $K_e = 100$ y $p = 50$. La entrada escalón sigue teniendo un valor final 5.

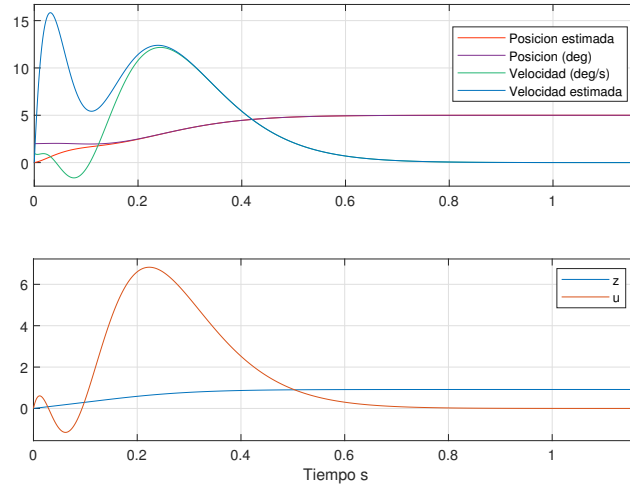
La figura 5.11b muestra, en la gráfica superior, la evolución de los estados del sistema así como la de los estados estimados. Puede observarse como, ahora el motor sí consigue alcanzar la posición angular 5 correspondiente al valor de referencia consignado por la entrada escalón.

En la gráfica inferior se ha representado el estado de la acción integral z junto con el valor de la entrada u . Es interesante notar, como el estado z alcanza un valor estacionario distinto de cero, cuando el motor ha alcanzado su valor estacionario.

Hemos prestado poca atención a u , el valor obtenido para la señal de entrada. Es importante recordar que



(a) Modelo del sistema con el estimador de estados



(b) Evolución de los estados y señal de entrada al motor

Figura 5.11: Realimentación de estados estimados para el motor ideal

dicho valor corresponde al voltaje V_{ref} con que debemos alimentar el motor. Conviene recordar que dicho valor no puede superar el valor $V_{max} = 12V$ en el sistema real. Ahora que tenemos un modelo completo, podemos estudiar el efecto de la posición de los polos —y, por tanto, de las ganancias K y L — tanto en la respuesta del sistema como en los valores que alcanzará la señal de entrada u .

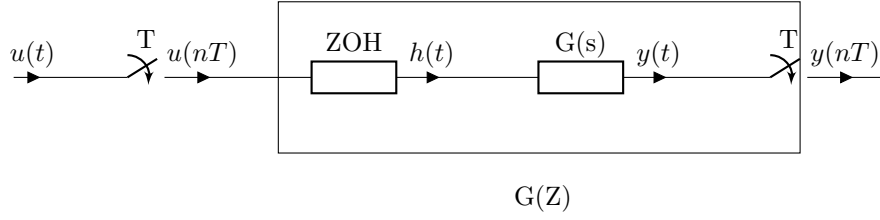
Así mismo podemos observar también el efecto que tiene el error en posición del motor, es decir, la diferencia entre la posición angular de su eje y el valor de referencia o valor final que debe alcanzar $\theta_r - \theta(t)$.

Si el sistema de control demanda valores mayores que V_{max} , esto no tiene efecto alguno en el sistema ideal, pero llevará a una saturación en la entrada del sistema real. En principio esto no debería tener mayores consecuencias en el motor real, que simplemente giraría a velocidad máxima hasta acercarse lo suficiente al valor de referencia de la posición como para que la acción de control deje de saturar la entrada, a partir de ahí, disminuiría el voltaje de entrada hasta alcanzar su posición final³.

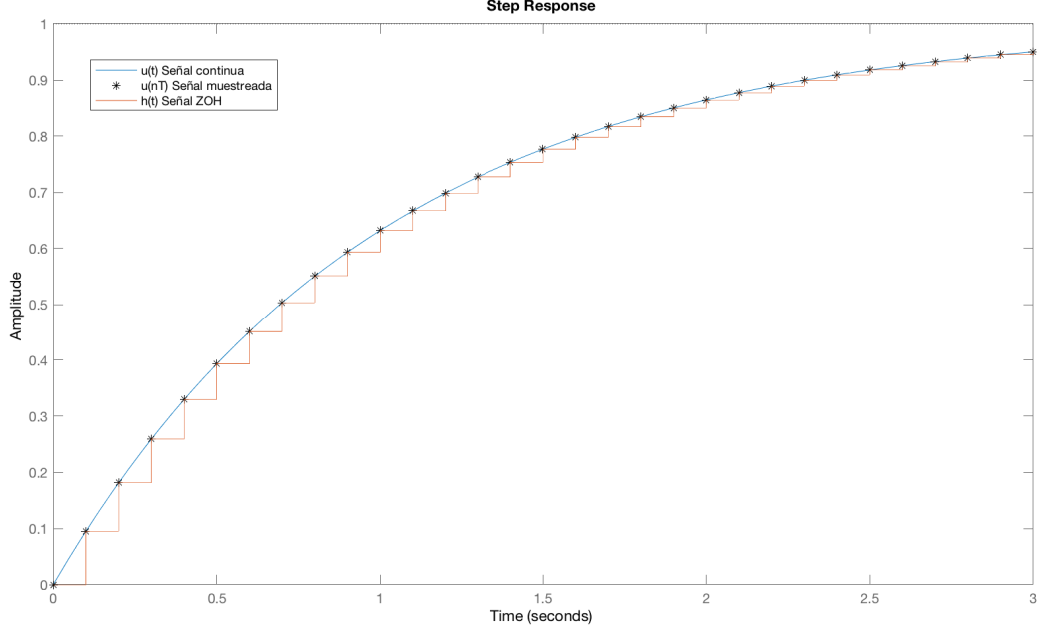
Práctica 4

1. Construir un modelo en Simulink, que incluya el motor ideal y un sistema de control por realimentación de estados estimados y acción integral.
2. Comprobar el efecto de la posición de los polos del sistema y el estimador sobre la velocidad de respuesta del sistema y el valor de la señal de control u . (incrementarlos o disminuirlos en potencias de 10)
3. Comprobar el efecto de la ganancia de la alimentación directa F_c .
4. Observar el efecto del error de posición inicial sobre la señal de entrada u .

³Esto no es completamente cierto debido a la presencia del control integral, que podría hacer que el motor oscilase. Hablaremos de ello más adelante.



(a) Esquema de una planta continua y su discretización mediante el uso de un retenedor de orden cero (ZOH) y muestreadores



(b) Comparación entre una señal continua $u(t)$, su versión muestreada $u(nT)$ y su aspecto $h(t)$ tras cruzar un ZOH

Figura 5.12: Sistema muestreador/retenedor de orden 0 (ZOH)

5.2.2. Discretización del controlador para control del motor real

Para terminar de diseñar el sistema de control, debemos tener en cuenta que el motor es controlado y observado por un sistema digital. Es decir, la señal de entrada (voltaje) que recibe el motor y la señal de salida (lectura de encoders) no se realiza de modo continuo, sino discreto. Marcado por el periodo de muestreo al que la placa ST Nucleo envía o recibe datos por sus pines. Podemos modelar el efecto del muestreo, empleando un retenedor de orden cero (ZOH) y un muestreador. La figura, 5.12a muestra un modelo de entrada y salida de una planta continua empleando retenedores. La señal de entrada $u(t)$ es muestreada con un periodo de muestreo T . Las muestras, que corresponden a los valores de la señal en periodos de tiempo regulares múltiplos del periodo $u(nT)$, son introducidas en el retenedor que, mantendrá el valor recibido hasta el siguiente instante de muestreo $(n+1)T$. La salida del retenedor es una señal continua a *escalones*. Un ejemplo de estas tres señales se muestra en la figura 5.12b.

Si construimos en Simulink, un control en variables de estado, y lo añadimos al sistema desarrollado para manejar el motor real a través de la placa ST Núcleo, tenemos que tener presente por tanto, que ésta no ve al motor como un sistema continuo sino como un sistema discreto con una entrada $u(nT)$ y una salida $y(nT)$, que equivaldría a los bloques ZOH, $G(s)$ y el muestreador, recuadrados en la figura 5.12a.

Deberíamos, por tanto obtener un modelo de la versión discreta del motor, tal y como la ve la placa. En general, un modelo en variables de estado discreto, se define como,

$$\mathbf{x}(n+1) = F\mathbf{x}(n) + G\mathbf{u}(n) \quad (5.3)$$

$$\mathbf{y}(n) = C\mathbf{x}(n) + D\mathbf{u}(n) \quad (5.4)$$

El papel de las matrices F , G , C y D es análogo al de las matrices A , B , C y D en el caso continuo. La diferencia fundamental es que ahora las variables no son función del tiempo sino del número de muestra n y

que la ecuación de estados relaciona los estados en la muestra $n + 1$ con los estados en la muestra n .

Si queremos relacionar un sistema continuo, con su versión discreta obtenida por muestreo con un periodo T , debemos considerar que los valores de las señales del sistema discreto (muestras) deben coincidir con las del sistema continuo para precisamente en los instantes de muestreo $t = nT$, es decir, $y(n) = y(t)$, $\forall t = nT, n = 0, 1, \dots$

Para el sistema continuo sabemos que podemos expresar sus estados y su respuesta a un estado inicial x_0 y una entrada $u(t)$ como,

$$x(t) = e^{A(t-t_0)}x_0 + \int_{t_0}^t e^{A(t-\tau)}Bu(\tau)d\tau \quad (5.5)$$

$$y(t) = Cx(t) + Du(t) \quad (5.6)$$

La versión discreta solo considerará los valores correspondiente a los instantes de muestreo nT . Supongamos que calculamos la evolución de los estados del sistema precisamente entre dos instantes de muestreo sucesivos, $t = (n + 1)T$, $t_0 = nT$,

$$x((n + 1)T) = e^{AT}x(nT) + \int_{nT}^{(n+1)T} e^{A((n+1)T-\tau)}Bh(\tau)d\tau \quad (5.7)$$

$$y(nT) = Cx(n) + Du(n) \quad (5.8)$$

Donde sustituimos la señal de entrada $u(t)$ por su valor tras atravesar el retenedor $h(\tau)$. Por otro lado, dados los límites de la integral, el retenedor estará dando como salida constante durante todo el periodo el valor de la variable u al comienzo del mismo: $h(\tau) = u(nT)$. Como se trata de un valor constante, podemos sacarlo fuera de la integral. El resultado sería

$$x((n + 1)T) = e^{AT}x(nT) + \left[\int_{nT}^{(n+1)T} e^{A((n+1)T-\tau)}Bd\tau \right] u(nT) \quad (5.9)$$

$$y(nT) = Cx(nT) + Du(nT) \quad (5.10)$$

Sin pérdida de generalidad podemos sustituir en las definiciones de las variables nT por n , puesto que el periodo de muestreo solo nos indica la relación de las muestras con el instante en que se tomaron. Además es fácil ver que la integral de la ecuación 5.9 es precisamente la convolución sobre un periodo de la matriz de transición de estados con un escalón unitario. Por otro lado, esto no tiene nada de sorprendente puesto que el retenedor convierte la señal de entrada en escalones de duración T . Podemos, por tanto, calcular la integral para un periodo,

$$\int_{nT}^{(n+1)T} e^{A((n+1)T-\tau)}Bd\tau = \int_0^T e^{A(T-\tau)}Bd\tau = e^{AT} \left[\int_0^T e^{-A\tau}d\tau \right] Bu(n) \quad (5.11)$$

Podemos reescribir las ecuaciones 5.9, 5.10 como,

$$x(n + 1) = e^{AT}x(n) + e^{AT}e^{AT} \left[\int_0^T e^{-A\tau}d\tau \right] Bu(n) \quad (5.12)$$

$$y(n) = Cx(n) + Du(n) \quad (5.13)$$

Podemos ahora relacionar los resultados obtenidos con los valores de las matrices F , G , C y D del sistema en variables de estado discreto (ecuaciones 5.3 y 5.4),

$$F = e^{AT} \quad G = e^{AT} \left[\int_0^T e^{-A\tau}d\tau \right] B \quad (5.14)$$

$$C = C \quad D = D \quad (5.15)$$

Si la matriz A es invertible, podemos calcular directamente la integral de la expresión para G ,

$$G = e^{AT}A^{-1} [I - e^{-AT}] B$$

En nuestro caso A es una matriz defectiva, pero eso hace fácil el cálculo de su exponencial:

$$e^{-A\tau} = \exp \left(- \begin{bmatrix} 0 & 1 \\ 0 & -p \end{bmatrix} \tau \right) = \begin{bmatrix} 1 & -(e^{p\tau} - 1)/p \\ 0 & e^{p\tau} \end{bmatrix}$$

Si calculamos la integral entre 0 y T ,

$$\int_0^T e^{-A\tau} d\tau = \begin{bmatrix} T & Tp - e^{Tp} + 1)/p^2 \\ 0 & (e^{Tp} - 1)/p \end{bmatrix}$$

Obtenemos finalmente,

$$G = e^{AT} \begin{bmatrix} T & Tp - e^{Tp} + 1)/p^2 \\ 0 & (e^{Tp} - 1)/p \end{bmatrix} B \quad (5.16)$$

Para controlar el motor real empleando realimentación de estados estimados, debemos sustituir el estimador continuo por su versión discreta, calculando los valores de las matrices F y G a partir de los valores de las matrices A y B y del periodo de muestreo T . Matlab suministra el comando `c2d` que obtiene directamente la versión muestreada de un sistema continuo definido en variables de estado mediante el comando `ss`. Así si el sistema continuo es: `sys = ss(A,B,C,D)` el discreto correspondiente para un periodo de muestreo T se obtiene como: `sysd = c2d(sys,T)`.

Discretización de los polos Debemos recalcular los valores de las ganancias K y L para emplearlos con el estimador discreto. Para ello, podemos trasladar los polos de continuo a discreto. Supongamos que queremos para el sistema continuo un polo de valor λ_i . Sabemos que la respuesta asociada a este polo está relacionada con una función exponencial, si muestreemos dicha respuesta:

$$y(t) \approx e^{\lambda_i t} \rightarrow y(nT) \approx e^{\lambda_i nT} \rightarrow y(n) \approx (e^{\lambda_i T})^n \quad (5.17)$$

$$\lambda_i \rightarrow \Lambda_i = e^{\lambda_i T} \quad (5.18)$$

Por tanto, podemos obtener los polos del sistema discreto (muestreado) Λ_i a partir de los polos del sistema continuo λ_i y el tiempo de muestreo T . Es interesante notar cómo el carácter estable o inestable del sistema no cambia al discretizarlo, si $\text{re}\{\lambda_i\} \leq 0$ entonces $|\Lambda_i| < 1$ que corresponde con la condición de estabilidad de un sistema discreto $\Lambda_i \leq 1 \Rightarrow \lim_{n \rightarrow \infty} \Lambda_i^n = 0$.

Las ganancias K y L para el sistema discreto pueden obtenerse, igual que el caso continuo empleando `acker` o `place`: Se diseñan la posición de los polos para la planta λ_{pi} y el observador λ_{oi} , se discretizan empleando la ecuación 5.18 y se introducen en `acker` junto con las matrices del sistema discretizado:

$$K = \text{acker}(F, G, [\Lambda_{p1} \dots \Lambda_{pn}])$$

$$L = (\text{acker}(F^T, C^T, [\Lambda_{o1} \dots \Lambda_{on}]))^T$$

Discretización de la acción integral. Debemos también discretizar la acción integral para incluirla en el controlador del motor. Si partimos de la ecuación de estado para la acción integral y la discretizamos,

$$\dot{z}(t) = C\mathbf{x}(t) - x_r \rightarrow \frac{z(n+1) - z(n)}{T} = C\mathbf{x}(n) - x_r \rightarrow z(n+1) = z(n) + TC\mathbf{x}(n) + Tx_r$$

Podemos emplear esta última ecuación directamente para construir el sistema ampliado,

$$\begin{bmatrix} \mathbf{x}(n+1) \\ z(n+1) \end{bmatrix} = \begin{bmatrix} F & 0 \\ TC & 1 \end{bmatrix} \cdot \begin{bmatrix} \mathbf{x}(n) \\ z(n) \end{bmatrix} + \begin{bmatrix} G \\ 0 \end{bmatrix} u(n) + \begin{bmatrix} 0 \\ -T \end{bmatrix} x_r \quad (5.19)$$

Podemos directamente emplear el sistema ampliado con la acción integral discreta para recalcular los valores de las ganancias $[K, k_i]$ empleando el mismo método descrito en el párrafo anterior.

Acción Directa Una última nota para el cálculo de la acción directa para el sistema muestreado. Hemos omitido su cálculo hasta ahora, ya que, si empleamos acción integral podemos asignarle un valor arbitrario (aunque razonable). Vamos de todos modos a recordar la expresión de la acción directa para un sistema continuo y como adaptarla para una versión discretizada dicho sistema. Si llamamos f a al valor de la ganancia de la acción directa necesaria para que la planta alcance el estado estacionario tendremos que,

$$\mathbf{x}_{ssD} = (F - GK)\mathbf{x}_{ssD} + Gfx_r \quad (5.20)$$

$$[I - (F - GK)] \mathbf{x}_{ssD} = Gfx_r \quad (5.21)$$

$$\mathbf{x}_{ssD} = [I - (F - GK)]^{-1} Gfx_r \quad (5.22)$$

$$y_{ssD} = C\mathbf{x}_{ssD} = x_r, (D = 0) \quad (5.23)$$

$$x_r = C[I - (F - GK)]^{-1} Gfx_r \quad (5.24)$$

$$C[I - (F - GK)]^{-1} Gf = I \quad (5.25)$$

$$f = [C[I - (F - GK)]^{-1} G]^{-1} \quad (5.26)$$

Es interesante observar que el resultado no coincide con el cálculo de la ganancia F_c para el caso continuo. De hecho, ambos sistemas, continuo y discretizado, alcanzarán el mismo valor de consigna pero por caminos distintos.

Si se emplea acción integral, es posible elegir libremente tanto F_c como f . Se puede entonces ajustar el valor de f para que las trayectorias de ambos sistemas coincidan. Así para valores dados de F_c y f , sabemos que,

$$y_{ss} = -C[A - BK]^{-1}BF_cx_r \quad (5.27)$$

$$y_{ssD} = C[I - (F - GK)]^{-1}Gfx_r \quad (5.28)$$

Buscamos asegurar que ambos valores (continuo y discretizado) en estacionario sean iguales $Y_{ss} = Y_{ssD}$. Si igualamos las expresiones de los estados estacionarios y despejamos f

$$-C[A - BK]^{-1}BF_cx_r = C[I - (F - GK)]^{-1}Gfx_r \quad (5.29)$$

$$f = -\left[C[I - (F - GK)]^{-1}G\right]^{-1}C[A - BK]^{-1}BF_c \quad (5.30)$$

$$(5.31)$$

Obtenemos una relación entre f y F_c

Práctica 5

1. Diseñar un sistema de control basado en realimentación de estados para el motor, empleando los parámetros K_e y p identificados en la práctica 2.
2. Estudiar el efecto de la posición de los polos y los valores de f en la respuesta obtenida para entradas escalón. Examinar el efecto sobre la entrada del sistema u .

5.3. Controlador discreto para el modelo completo del motor y el motor real

Finalmente, podemos construir un modelo de Simulink que incluya el modelo del motor desarrollado en la sección 5.1 y el controlador discretizado que acabamos de describir en la sección anterior.

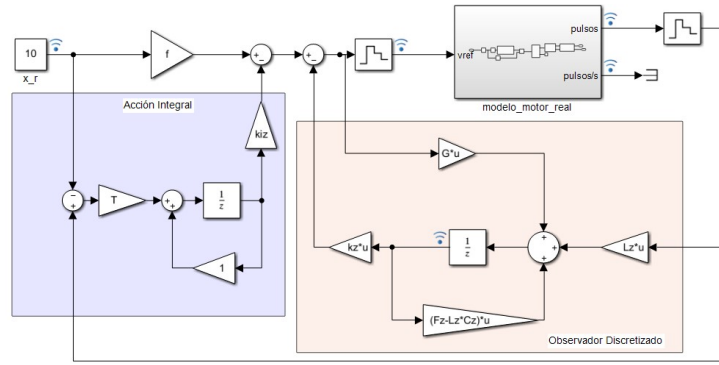
La figura 5.13a muestra el modelo completo.

El bloque marcado como `modelo_motor_real` contiene el modelo del motor completo (ver figura 5.7a) a la entrada y salida del bloque, se han añadido un par de bloques ZOH para modelar el muestreo del sistema de adquisición y control propio de la placa ST Nucleo. Recuadrado en rojo aparecen los bloques correspondientes al modelo de motor discretizado, con el que se ha construido el estimador. El bloque marcado en azul corresponde a la acción integral discretizada.

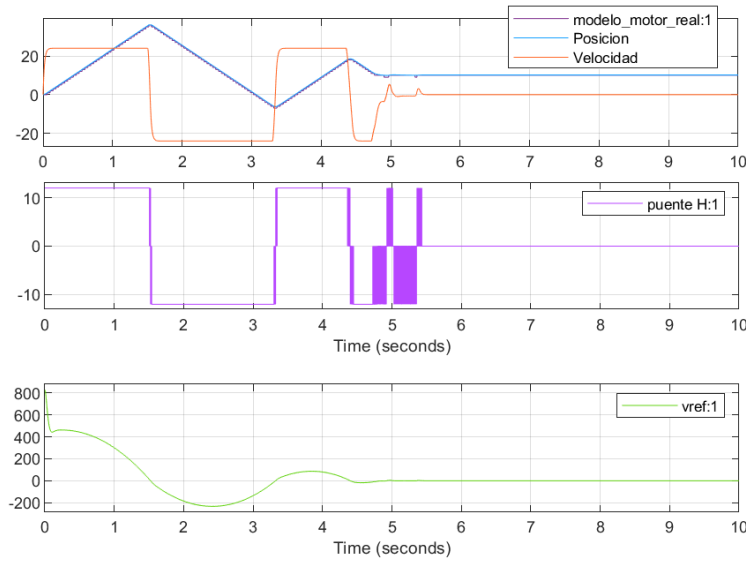
La figure 5.13b muestra la respuesta del sistema con los siguiente parámetros:

- Los parámetros del motor se han fijado como $K_e = 100$ y $p = 50$
- Los polos del sistema continuo se han seleccionado como:
 1. $P = [-15 - 15 - 20]$. Polos para realimentación de estados, el ultimo corresponde a la acción integral
 2. $PL = [-30 - 30]$. Polos para la realimentación del error de la salida para el estimador
- F_c calculada para asegurar que el sistema alcanza el valor de referencia $F_c = (C(BK - A)^{-1}B)^{-1}$. (aunque no sería necesario).
- Tiempo de muestreo $T = 0,0001s$
- $x_r = 10^\circ$
- Todos los parámetros correspondientes al sistema discretizado se han obtenido a partir de las ecuaciones descritas en la sección anterior f se ha calculado a partir de F empleando la ecuación 5.31.

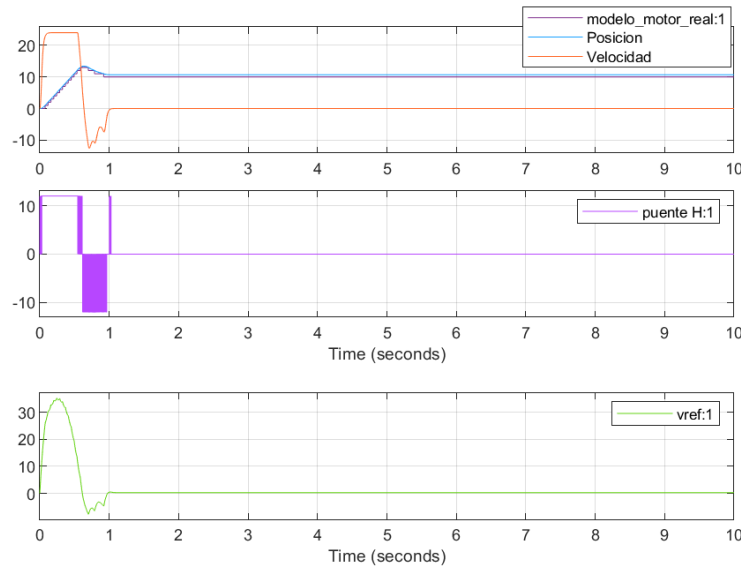
Si observamos la salida del sistema (fig. 5.13b) vemos que el motor alcanza la posición final tras realizar un par de oscilaciones amplias. Lo que no corresponde con el resultado que obtendríamos si empleáramos un modelo ideal del motor. La razón la podemos obtener comparando el gráfico en que se representa la variable $vref$, —que corresponde al valor de voltaje de entrada calculado por el sistema de control— con el gráfico en que se representa la variable $punteH : 1$ que nos da el voltaje real suministrado por el puente en H del modelo.



(a) Modelo de motor completo con control discretizado

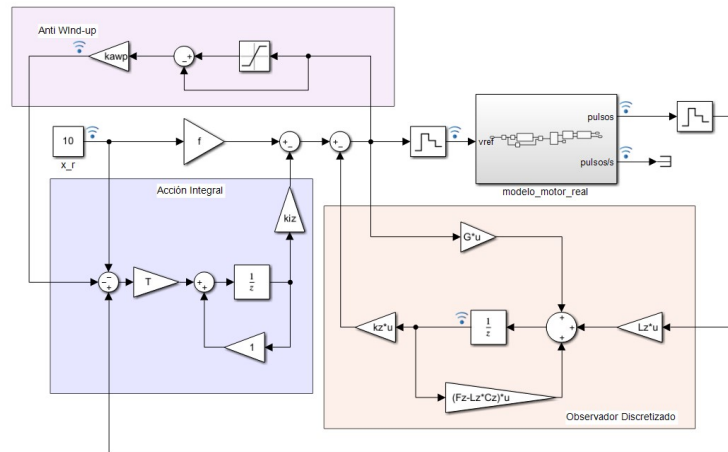


(b) Respuesta del sistema f calculado a partir de F

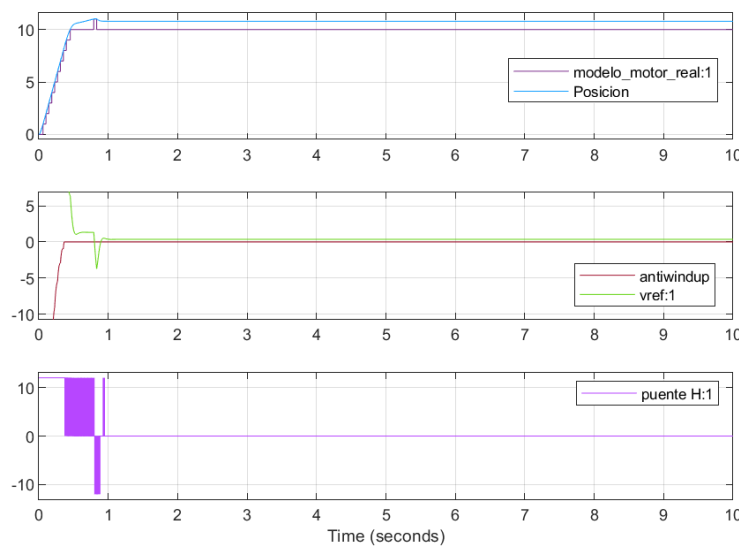


(c) Respuesta del sistema $f = 0$

Figura 5.13: Modelo completo del motor con control discretizado por realimentación de estados estimados, acción integral y acción directa



(a) Modelo de motor completo con control discretizado y anti wind-up



(b) Respuesta del sistema f calculado a partir de F

Figura 5.14: Modelo completo del motor con control discretizado por realimentación de estados estimados, acción integral, acción directa y Anti wind-up.

El puente solo puede suministrar un voltaje máximo de 12V, sin embargo, el controlador demanda un voltaje inicial mucho más alto. El resultado es que el motor recibe un voltaje constante de 12 V durante los primeros 1.5s. Además la posición supera con mucho el voltaje de consigna. La razón de esta sobreoscilación tan grande, se debe a la acción integral: su contribución al error crece durante mucho más tiempo del que sería deseable, ya que el motor, alimentado con 12 voltios, avanza mucho más despacio de lo previsto, y el error integral sigue creciendo mientras el error no es cero. Cuando se alcanza el valor de referencia, la acción integral sigue obligando al motor a avanzar en la misma dirección hasta que cambia el signo de la integral. Es fácil ver que el efecto resultante —conocido como *wind-up* de la acción integral, genera una oscilización innecesaria en el sistema que, para ciertos valores de los parámetros (posiciones de los polos y valor de f) podría llegar a desestabilizarlo.

Para paliar en parte este efecto, una primera solución es eliminar la acción directa $f = 0$. La figura 5.13c muestra los resultados obtenidos, para el mismo caso anterior pero eliminando la acción directa. Es claro como ahora el sistema se estabiliza antes y se reducen las oscilaciones.

Anti wind-up Una solución más robusta a los problemas de *Wind-up*, consiste en desconectar o atenuar la acción integral mientras el actuador está saturado, es decir mientras el valor de la entrada demandado por el controlador supera el valor que éste puede dar. La figura 5.14a muestra una posible construcción de dicho mecanismo, remarcado en color rosa. En este caso, no se desconecta la acción integral, sino que se altera la entrada del error de posición al integrador añadiendo un nuevo término proporcional a la diferencia entre la entrada u calculada por el controlador y valor máximo de la salida. El valor de la constante de proporcionalidad —*kamp* en la figura— se ajusta habitualmente en función de las características que se quieran dar a la salida.

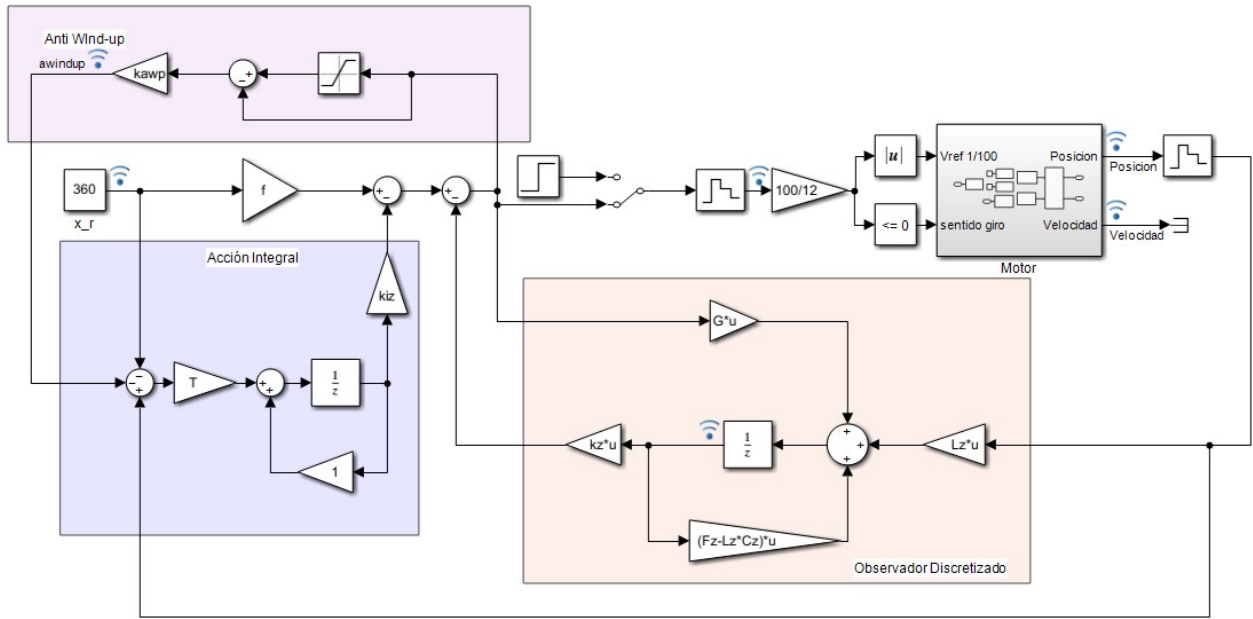


Figura 5.15: Sistema completo para alimentar el motor real, empleando el esquema de Simulink (fig. 3.2) para manejar el motor real. Los bloques añadidos antes del motor permiten pasar de tensión en voltios a señal de PWM

Práctica6

1. Construir los bloques de Simulink del estimador acción directa y acción integral discreta y añadirlos al modelo de motor completo desarrollado en la práctica 3.
2. Estudiar la respuesta del sistema para los sistemas de control diseñados en la práctica 5. Comprobar el efecto de eliminar la acción integral $K_i z = 0$ o la acción directa $f = 0$
3. Añadir al modelo un bloque anti wind-up, analizar la salida para distintos valores de la constante $kamp$.
4. Trasladar los bloques del sistema de control desarrollado al esquema de Simulink empleado para manejar el motor real (fig 5.15). Comprobar que es capaz de mover el motor desde una posición inicial $\theta_0 = 0$ hasta una posición final de referencia θ_r

Bibliografía

- [1] Devantech. RB-DEV-40 Devantech 12V, 30:1 Gear Motor w/ Encoder. https://www.robotshop.com/media/files/pdf/datasheet-emg30_1.pdf, 1200.
- [2] STMicroelectronics. UM1724 User manual. STM32 Nucleo-64 boards (MB1136). <https://www.st.com/en/evaluation-tools/nucleo-f411re.html#>, April 2019.
- [3] STMicroelectronics. STSPIN840. Compact dual brushed DC motor driver. <https://www.st.com/en/motor-drivers/stspin840.html>, May, 2018.
- [4] STMicroelectronics. UM2393. User Manual. Getting started with the X-NUCLEO-IHM15A1 dual brush DC motor driver expansion based on STSPIN840 for STM32Nucleo. <https://www.st.com/en/motor-drivers/stspin840.html#tools-software>, May, 2018.